

Inbetriebnahme und Programmierung eines autonomen Labyrinth-Roboters am Beispiel des Pololu 3Pi+ 2040

Kilian Halbritter
Matrikelnr.: 2244309

Betreuer: Prof. Dr.-Ing. Alexander Biedermann

12. November 2025

Zusammenfassung

Inbetriebnahme, Analyse der Sensorik und Programmierung einer Micromouse auf Basis des Pololu 3Pi+ 2040 und dem Time of Flight Sensor OPT3101.



Abbildung 1: Micromouse

Inhaltsverzeichnis

1	Einsatz einer Micromouse im Hochschulwesen in der technischen Informatik	3
2	Der Roboter: Hardware, Aufbau und Komponenten	4
2.1	Zusammenbau und Modifikationen	4
2.2	Mechanischer Aufbau und Antriebssystem	7
2.3	Sensorik des Roboters: Aufbau und Funktionsweise	10
2.4	OLED-Display (SH1106)	17
2.5	Kommunikationsschnittstellen und Pinbelegung	18
3	Programmierung und Inbetriebnahme	20
3.1	Einrichtung der Entwicklungsumgebung	20
3.2	Programmierung der Aktoren	24
3.3	Auslesen und Verarbeiten der Sensoren	28
3.4	Ansteuerung des OLED-Displays	36
3.5	Interaktion und Debugging	37
4	Anwendungsfall: Das Labyrinth	39
4.1	Regelwerk des offiziellen Micromouse-Contest von IEEE	39
4.2	Aufbau und Fertigung des Labyrinths	44
4.3	3D-Drucker als Medium zur Realisierung der Kunststoffkomponenten . . .	45
5	Fernziele	48
	Anhang	I
A	GPIO-Pinout des Pololu 3pi+ 2040	I
B	Bohrplan Labyrinth	IV
C	Schaltplan 3pi+ 2040 Control Board	V
D	Laufzeitsensor (Time of Flight) OPT3101	X
E	Schaltplan Pololu OPT3101	X
F	SH1106G OLED-Display	XI
G	RP2040 Datenblatt	XI
H	GitHub-Bibliotheken	XI
I	3D-Modelle für die Labyrinthwände und -ecken	XI
J	Weiterführende Links zur RP2040-Programmierung in Python	XI
K	Weiterführende Links zur RP2040-Programmierung in C++	XII

Abkürzungsverzeichnis

Abkürzung	Beschreibung
ACK	Acknowledge (dt. für Bestätigung)
API	Application Programming Interface (dt. für Anwendungsschnittstelle)
bpm	beats per minute (dt. für Schläge pro Minute)
dps	degrees per second (dt. für Grad pro Sekunde)
FDM	Fused Deposition Modeling (dt. für Schmelzschichtung)
FET	Feldeffekttransistor
FoV	Field of View (dt. für Sichtfeld)
GPIO	General Purpose Input/Output (dt. für universelle Ein-/Ausgabe)
I²C	Inter-Integrated Circuit
IEEE	Institute of Electrical and Electronics Engineers
IMU	Inertial Measurement Unit (dt. für Trägheitsmesseinheit)
LED	Light Emitting Diode (dt. Leuchtdiode)
LSB	Least Significant Bit (dt. für niedrigstwertiges Bit)
MDF	Mitteldichte Faserplatte
MSB	Most Significant Bit (dt. für höchstwertiges Bit)
OSB	Oriented Strand Board (dt. für Grobspanplatte)
PLA	Poly lactide (auch Polymilchsäuren)
PWM	Pulsweitenmodulation
SAC	Student Activities Committee
SCL	Serial Clock (dt. für serielle Taktleitung)
SDA	Serial Data (dt. für serielle Datenleitung)
ToF	Time of Flight (dt. für Flugzeit, Laufzeit)

1 Einsatz einer Micromouse im Hochschulwesen in der technischen Informatik

Autonome Robotersysteme sind in Forschung und Lehre an Hochschulen unverzichtbare Werkzeuge, um Schlüsseltechnologien wie Künstliche Intelligenz, Robotik, Sensorik und Steuerungstechnik praxisnah zu erforschen und zu vermitteln. Aschaffenburg, 2024 Bielefeld, 2024 Durch praxisnahe Projekte und Laboreinheiten wird Studierenden die Möglichkeit gegeben, reale Probleme der Sensorik, Steuerung und Navigation durch das Labyrinth kennenzulernen und selbstständig Lösungen zu entwickeln. Diese Art des praxisorientierten Arbeitens ermöglicht es, theoretisches Wissen in praxisnahen Anwendungen umzusetzen und in Echtzeit ausprobieren zu können in einem sicheren Rahmen, der Fehler verzeiht. Es werden außerdem die Fähigkeiten des logischen Denkens, Systemabläufe, Umsetzungsvermögen trainiert. In Anwendungen mit dem Roboter erlernt man zusätzlich den Umgang mit Sensorik, dessen Probleme, und wie Sensoren funktionieren. Man lernt,

wie die Sensoren in den Abläufen sinnvoll integriert werden können, um ein gewünschtes Ergebnis zu erzielen.

Ein spezielles Beispiel hierfür ist das Micromouse-Projekt. Die Micromouse ist ein kompakter, autonomer Roboter, dessen Hauptaufgabe darin besteht, ein Labyrinth mithilfe einer Kombination aus Sensoren und Antriebssystemen eigenständig zu durchqueren. Der vermutlich erster Wettbewerb wurde 1977 vom Institute of Electrical and Electronics Engineers (IEEE) in einer Zeitschrift aufgerufen. Christiansen, 2014 Inzwischen finden jedes Jahr weltweit Micromouse-Wettbewerbe statt.

Der Roboter soll der Hochschule als Lernplattform dienen, um zentrale Themen der Elektro- und Regelungstechnik zu vermitteln. Man kann sogar noch weiterdenken und das Thema maschinelles Lernen hinzufügen, sodass der Roboter selbst lernt, sich zu verbessern, um das Labyrinth schneller zu lösen. Maschinelles Lernen wird jedoch nicht Teil dieser Studienarbeit sein.

In dieser Studienarbeit wird der Schwerpunkt auf die Sensorik und Ansteuerung der Application Programming Interface (API) gelegt. Außerdem wird die hardwaretechnische Umsetzung und Kalibrierung der Sensoren näher betrachtet.

Für die Umsetzung wird der Roboter Pololu 3Pi Plus 2040, aus dem Hause Pololu, verwendet. Der Roboter basiert auf einem Raspberry Pi Pico (RP2040-Mikrocontroller). Die Sensoren basieren auf Pololu-Produkten. Der Roboter ist mit umfassender Sensorik ausgestattet, die bei der Projektierung optional je nach Bedarf eingesetzt werden können. Es ist ein zusätzlicher externer Time-of-Flight-Sensor verbaut. Für das Projekt gibt es zwei verschiedene Versionen des Roboters, eine Standardvariante und eine Hypervariante. Sie unterscheiden sich im Wesentlichen durch die unterschiedlich mögliche Höchstgeschwindigkeit. Die wesentlichen unterschiedlichen Merkmale werden in Unterabschnitt 2.2 genauer erklärt.

2 Der Roboter: Hardware, Aufbau und Komponenten

2.1 Zusammenbau und Modifikationen

Die gelieferten Roboter wurden in zwei unterschiedlichen Zuständen bestellt: Ein Teil kam vollständig vormontiert an, der andere Teil als Bausatz in Einzelteilen. Die Montage der Kits erfolgte gemäß den Anweisungen des Herstellers, wobei besonderer Wert auf die Gewährleistung einer zuverlässigen elektrischen Verbindung aller Komponenten gelegt wurde.

Vor dem Zusammenbau des Roboters ist eine sorgfältige Überprüfung der Platinenfunktion essenziell, um mögliche Defekte frühzeitig zu identifizieren und spätere Probleme zu vermeiden. Dazu wird die Platine zunächst mit einem Computer verbunden, wobei zwei grüne leuchtende Dioden (LEDs) aufleuchten sollten, um die grundsätzliche Betriebsbereitschaft anzuzeigen. Im nächsten Schritt erfolgt ein Funktionstest der Programmierschnittstelle. Hierzu wird die Platine mit einem getesteten, funktionierenden Programm beschrieben, das eine einfache Aktion ausführt, beispielsweise das Blinken der auf der Platine verbauten orangen LED. Dieses Verfahren dient dazu, sowohl die Kommunikation zwischen Computer und Platine als auch die Funktionsfähigkeit der Schnittstelle zu validieren.

Im Verlauf des Projekts stellte sich nach dem mechanischen Zusammenbau des Roboters heraus, dass die Programmierschnittstelle einer Platine defekt war. Eine Ersatzplatine wurde beim Hersteller bestellt. Der Roboter war dadurch vorübergehend außer Betrieb.

Eine fehlerfreie elektrische Verbindung ist wichtig, um den zuverlässigen Betrieb des Roboters sicherzustellen. Daher ist eine präzise Löttechnik von entscheidender Bedeutung, um eine saubere und langlebige Verbindung zwischen den Bauteilen zu gewährleisten.

Für die Lötarbeit wurde bewusst bleifreies Lötzinn verwendet, um gesundheitsschädigende Risiken zu minimieren. Bleihaltiges Lötzinn zeichnet sich zwar durch hervorragende Schmelz- und Fließeigenschaften aus und wurde bis 2006 weit verbreitet eingesetzt, jedoch ist es aufgrund seiner toxischen Eigenschaften innerhalb der Europäischen Union seit 2006 im Rahmen der RoHS-Richtlinie (Restriction of Hazardous Substances) COX und DRYs, 2003 für Standardanwendungen verboten. Bleifreies Lötzinn hat im Vergleich dazu eine höhere Schmelztemperatur von etwa 220 °C, was eine Löttemperatur von mindestens 320 °C erforderlich macht, um optimale Ergebnisse zu erzielen. Eine Faustregel ist das Löten von 100 °C über der Schmelztemperatur des Lötzinns. Reichelt, 2017

Ein besonderes Augenmerk wurde auf das Löten von Bauteilen mit großen Kupferflächen gelegt, beispielsweise an Masseflächen (Ground). Diese Flächen leiten Wärme durch ihre hohe thermische Leitfähigkeit stark ab, was höhere Löttemperatur erforderlich macht. Hier wurde die Temperatur entsprechend angepasst, um sicherzustellen, dass ausreichend Wärme für eine vollständige Verbindung bereitgestellt wird. Gleichzeitig wurde darauf geachtet, dass weder die Bauteile noch die Leiterplatte durch übermäßige Hitze beschädigt wurden. Eine zu hohe Löttemperatur kann zu thermischen Schäden führen, während eine zu niedrige Temperatur keine zuverlässige elektrische Verbindung ermöglicht.

Zur Verbesserung der Fließeigenschaften des Lötzinns und zur Sicherstellung einer optimalen Verbindung wurde zusätzlich Flussmittel verwendet. Dieses erleichtert die Benetzung der Löttoberfläche mit Lötzinn und sorgt für eine gleichmäßige Verteilung des Lötzinns. Nach dem Löten wurden alle Verbindungen sorgfältig unter einem Vergrößerungsglas kontrolliert, um mögliche Fehler wie kalte Lötstellen, Lötbrücken oder unzureichende Verbindungen zu identifizieren und zu beheben.

Für den Distanzsensor OPT3031 musste ein spezieller Adapter entwickelt und gefertigt werden. Der Roboter bietet zwar ungenutzte Schnittstellen und Löt pads für die Integration zusätzlicher Hardware, jedoch entsprachen die verfügbaren Pins auf der Roboterplatine nicht der erforderlichen Pinbelegung des Sensors. Der OPT3101 benötigt folgenden Anschlüsse: GND, eine Versorgungsspannung von 2,5 V bis 5,5 V sowie die I²C-Schnittstellenpins SDA und SCL. Da die vorhandenen Pins auf der Roboterplatine nicht in der passenden Anordnung vorlagen, war es nicht möglich, den Sensor direkt über Standard-Pinheader in der Mitte der Platine zu verbinden. Eine fehlerhafte Verbindung oder Verpolung hätte zu irreversiblen Schäden am Sensor oder an der Roboterplatine führen können. Ohne Schutzschaltung wie eine Eingangsdiode kann die Spannung Bauteile zerstören. Für die I²C-Schnittstelle wurden der GPIO Pin 4 und Pin 5 festgelegt und verwendet.

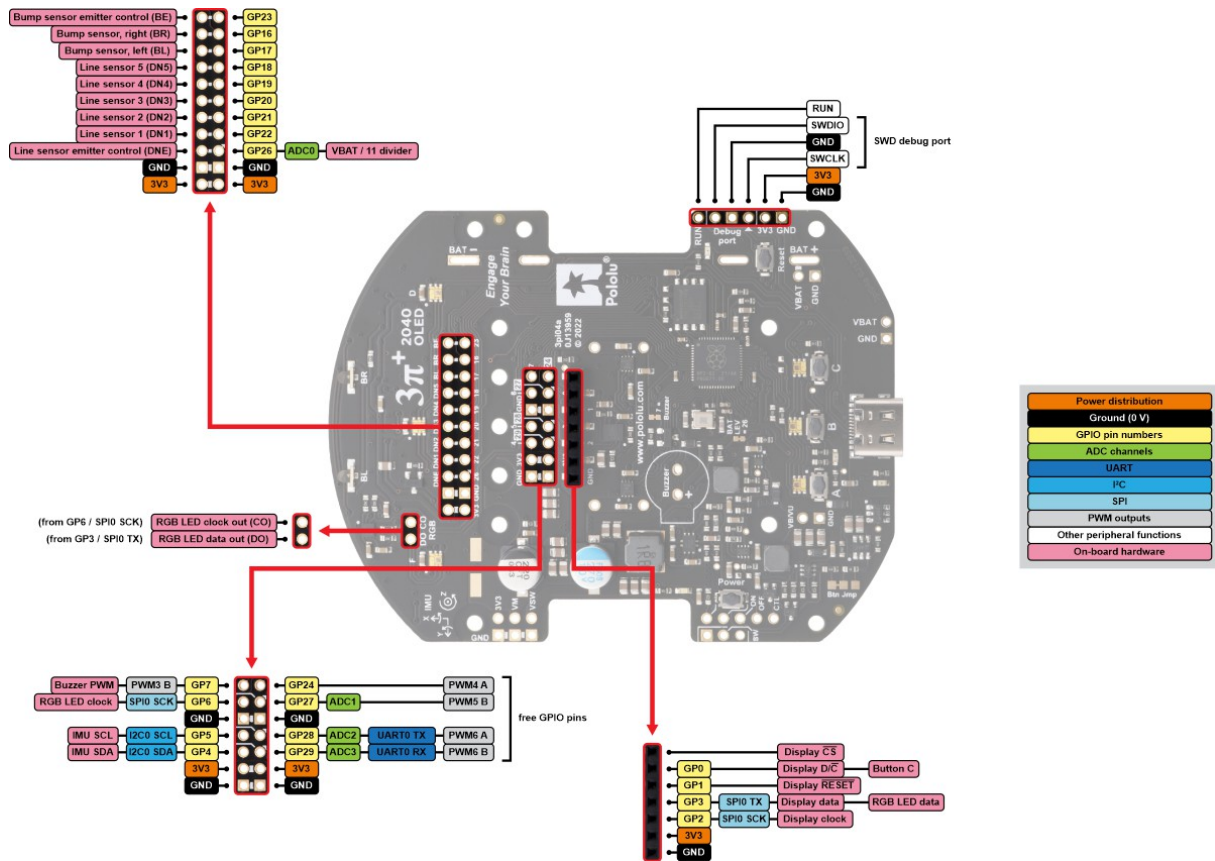


Abbildung 2: Pinout der Platine des Roboters

Um diese Problematik zu lösen und eine sichere Verbindung herzustellen, wurde ein Adapter konstruiert, der die korrekte Pinbelegung realisiert. Aus praktischen Gründen wurde der Sensor nicht fest verlötet, sondern steckbar ausgeführt, um einen einfachen und werkzeuglosen Austausch im Falle von Defekten oder für den Transport zu ermöglichen. Diese modulare Lösung bietet zudem Flexibilität, da die Pins für alternative Hardwareanwendungen verwendet werden können, sobald der Sensor entfernt wurde.

Der Adapter invertiert die Pinanordnung elektrisch, was bedeutet, dass die Verbindungen wie folgt umgesetzt wurden: Pin 1 der unteren Male-Pins wird mit Pin 4 der oberen Female-Pins verbunden, Pin 2 mit Pin 3, Pin 3 mit Pin 2 und Pin 4 mit Pin 1. Um Verwechslungen und Fehler zu vermeiden, wurde der Adapter entsprechend beschriftet, da die Pinbelegung nicht mehr direkt mit den Angaben im Datenblatt des Sensors übereinstimmt. Diese Beschriftung stellt sicher, dass auch bei späteren Wartungen oder Anpassungen die korrekte Orientierung eingehalten wird.

Eine zusätzliche Herausforderung stellte die Positionierung des Sensors dar. Die relevanten Pins auf der Roboterplatine sind nicht mittig angeordnet, was zu einem Versatz in der Montage führt. Da jedoch für die optimale Funktionalität eine zentrale Ausrichtung des Sensors erforderlich ist, wurde dieser Offset konstruktiv durch die Gestaltung eines passenden Adapters kompensiert. Ein gewisser Toleranzbereich wurde dabei in Kauf genommen, solange der Sensor optisch erkennbar mittig positioniert ist. Diese zentrale Lage ist essenziell, da der Sensor zur Abstandsmessung gegenüber Hindernissen dient. Befindet sich der Roboter beispielsweise mittig zwischen zwei parallelen Wänden, soll er auf beiden Seiten denselben Abstand erfassen können.

Für die Leitungsführung wurden Drähte mit einem Querschnitt von 0,32 mm² ver-

wendet, was für die geringe Stromaufnahme des Sensors vollkommen ausreichend ist. Um die mechanische Stabilität des Adapters zu gewährleisten, wurden die Verbindungen mit Schmelzklebstoff fixiert und abschließend mit Schrumpfschlauch ummantelt. Diese Konstruktion erlaubt es dem Adapter, den Sensor eigenständig zu tragen, ohne dass zusätzliche Halterungen erforderlich sind. Die Adapter wurden vor dem Einbau auf die richtige Kontaktierung mithilfe eines Durchgangsprüfers geprüft.

2.2 Mechanischer Aufbau und Antriebssystem

Aufbau Encoder

Der Roboter Pololu 3pi Plus 2040 ist als kompakte Einplatinenkonstruktion konzipiert und wird von zwei motorisierten Achsen angetrieben. Seine Abmessungen betragen laut Hersteller 97 mm in der Länge, 96 mm in der Breite und 36 mm in der Höhe. Mit dem zusätzlichen Distanzsensor OPT3101 steigt die Gesamthöhe des Roboters leicht an. Im Rahmen dieser Arbeit kommen zwei Varianten des Roboters zum Einsatz: die Standard- und die Hyper-Edition. Diese beiden Varianten unterscheiden sich in der mechanischen Übersetzung der Motoren auf die Antriebsachsen. Die Standard-Version ist mit einer Übersetzung von 30:1 (genau 29,86:1) ausgestattet, während die Hyper-Version eine Übersetzung von 15:1 verwendet. Die geringere Übersetzung der Hyper-Edition führt zu einer höheren Endgeschwindigkeit von bis zu 4 m/s, im Vergleich zu 1,5 m/s bei der Standard-Edition. Aufgrund der unterschiedlichen Übersetzungen weist die Hyper-Edition zudem eine umgekehrte Drehrichtung auf, was bei der Programmierung durch eine Invertierung der Steuerbefehle berücksichtigt werden muss. Bei Verwendung der Standardbibliothek lässt sich diese Invertierung unkompliziert in der Initialisierung vornehmen, sodass der Code im Anschluss korrekt arbeitet. Der Roboter lässt sich grob in zwei Hauptbestandteile unterteilen: das Gehäuse und die Elektronikplatine. Das Gehäuse ist schlicht gestaltet und dient hauptsächlich als Halterung für die Sensoren und die Platine. An der Vorderseite des Gehäuses befinden sich Halterungen für die Stoßsensoren, die mechanische Hindernisse erkennen. Die Elektroplatine enthält die wesentlichen Komponenten, darunter die Motorsteuerung, die Sensorik und Schnittstellen für die externe Programmierung. Der Durchmesser der Antriebsräder ist relevant, um sich die zurückgelegte Strecke errechnen zu können. Die Antriebsräder des Roboters haben einen Durchmesser von 32 mm, was für die Berechnung der zurückgelegten Strecke relevant ist. Jedes Rad wird unabhängig angesteuert, was eine präzise Steuerung ermöglicht, einschließlich Drehbewegungen auf der Stelle. Für zusätzliche Stabilität verfügt der Roboter über eine passive Kunststoffkugel als drittes Auflageelement im hinteren Bereich. Diese „geschleppte“ Kugel verhindert, dass das Gehäuse beim Fahren den Boden berührt, ohne eine Richtungskontrolle in Form einer Lenkung auszuüben.

Der Encoder ist aufgeteilt in Encoder-Magnetscheibe und Halleffekt-Sensor. Die magnetisch polarisierte Magnetscheibe sitzt auf der Verlängerung des Motorschafts. Ein Encoder detektiert Rotation und wandelt dies in ein Signal um. Durch dieses Signal und der bekannten Auflösung kann die Position oder Strecke gemessen werden, die der Motor gedreht ist. Da die Encoder-Scheibe direkt am Motor befestigt ist, muss man, um auf eine Umdrehung des Rades zu schließen, die Übersetzung des Getriebes mit berücksichtigen. Dabei ist es wichtig, dass die Übersetzung exakt ist und keine Toleranz besitzt. Die Encoder-Scheibe hat 6 Pole und somit eine Auflösung von 12 CPR (Clicks Per Rotation). Es ist aber unbedingt darauf zu achten, dass dies auf den Motor bezogen ist, und nicht auf eine Umdrehung des Rads. Um die Zählungen pro Radumdrehung zu berechnen, wird

der Getriebefaktor mit der Encoder-Auflösung multipliziert. (Standard Edition 29,86; Hyperedition 15). Beispielsweise ergibt eine Standardübersetzung von 29,86:1 eine Auflösung von $29,86 \cdot 12 \approx 358,3$ Clicks/Rotation für das Rad. Der Encoder funktioniert aber nur im Ganzen mit dem zusätzlichen Halleffekt-Sensor. Pololu, 2024c, S. 6.4 So kommt man auf eine Auflösung am Rad von 1° pro Click oder am Motor von 30° pro Click. Zur Auswertung der Encodersignale werden vier GPIO-Pins des Controllers genutzt:

- **GP8:** Kanal A des rechten Encoders
- **GP9:** Kanal B des rechten Encoders
- **GP12:** Kanal A des linken Encoders
- **GP13:** Kanal B des linken Encoders

Pololu, 2024c, S. 6.4

Motoransteuerung und Antrieb

Im Roboter sind Gleichstrommotoren verbaut, die über eine Pulsweitenmodulation (PWM) gesteuert werden. Dies ist eine etablierte Methode zur effizienten und verlustarmen Motorsteuerung. PWM ermöglicht es, die Leistung eines Motors zu regeln, ohne die Spannung direkt zu verändern. Die PWM-Ansteuerung ist besonders effizient, da der Feldeffekttransistor (FET) im PWM-Betrieb fast ausschließlich in den Zuständen „an“ oder „aus“ arbeitet, in denen der Leistungsverlust minimal ist, was sich in einer geringeren thermischen Belastung und höherem Wirkungsgrad zeigt. Dabei wird ein GPIO-Pin verwendet, der über einen FET den Motor mit dem PWM-Signal antreibt. Durch die Konfiguration der PWM-Signale lässt sich die Geschwindigkeit präzise steuern. Ein variabler Duty-Cycle des PWM-Signals legt fest, wie lange der GPIO-Pin pro Zyklus im „EIN“-Zustand bleibt (On-Time) und wie lange er auf „AUS“ ist (Off-Time). Je länger die On-Time im Vergleich zur Off-Time ist, desto mehr Leistung wird dem Motor zugeführt, was zu einer höheren Drehzahl führt. Um eine geschmeidige Motorsteuerung zu erreichen und hörbare Störgeräusche zu vermeiden, wird eine hohe PWM-Frequenz verwendet. Dadurch entstehen keine spürbaren Vibrationen, und der Motorlauf wirkt gleichmäßig. Die Auswahl einer geeigneten PWM-Frequenz und eines Duty-Cycle-Bereichs ermöglicht es, Geschwindigkeit und Leistung des Motors zwischen einem minimalen und maximalen Wert effizient und stufenlos zu regulieren. Die PWM-Ansteuerung ist besonders effizient, da der FET im PWM – Betrieb fast ausschließlich in den Zuständen „an“ oder „aus“ arbeitet, in denen der Leistungsverlust minimal ist. In folgender Grafik (Abbildung 3) ist die Darstellung einer Periode als PWM-Signal beispielhaft dargestellt.

$t_1 = \text{On-Time}$
 $T = \text{Periodendauer}$
 $T - t_1 = \text{Off-Time}$

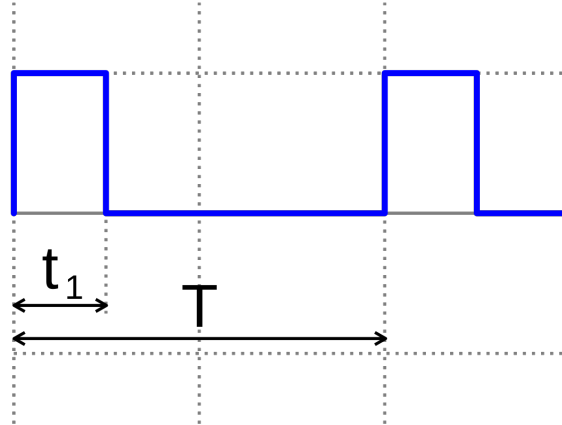


Abbildung 3: Darstellung einer Pulsweitenmodulation
 Krüger, [2008](#)

Die Räder des Roboters sind direkt auf den Wellen der Getriebe montiert. Die Getriebe sitzen unmittelbar vor den Motoren und sind direkt mit den Motorachsen verbunden. Die verwendeten Motoren sind Gleichstrommotoren, die durch ein PWM-Signal angesteuert werden. Diese Methode ermöglicht eine präzise Steuerung der Geschwindigkeit und Drehrichtung der Motoren. Die Getriebe sind so ausgelegt, dass sie die hohe Drehzahl der Motoren in eine niedrigere Drehzahl umwandeln, wodurch gleichzeitig das Drehmoment erhöht wird. Diese Übersetzung ist entscheidend für die Fortbewegung des Roboters, da sie genügend Kraft bereitstellt, um Reibung, das Eigengewicht und präzise Manöver auszuführen. Die Motoren werden mit einer Betriebsspannung von 6 V versorgt. Pololu, [2024b](#), Specs In Tabelle 2 werden genauere Spezifikationen aufgezählt.

Tabelle 2: Technische Spezifikationen des Motors

Abmessungen	
Größe:	10 × 12 × 25 mm
Gewicht:	9,5 g
Wellendurchmesser:	3 mm
Allgemeine Spezifikationen	
Getriebeübersetzung:	29,86:1
Leerlaufdrehzahl bei 6V:	720 U/min
Leerlaufstrom bei 6V:	0,07 A
Blockierstrom bei 6V:	0,67 A
Blockierdrehmoment bei 6V:	0,33 kg · cm
Max. Ausgangsleistung bei 6V:	0,57 W
Encoder:	Encoder-kompatibel (verlängerte Motorwelle)
Motortyp:	0,67 A Blockierstrom bei 6 V (MP 6 V)
Leistungsdaten bei maximalem Wirkungsgrad	
Max. Wirkungsgrad bei 6V:	33 %
Drehzahl bei max. Wirkungsgrad:	510 U/min
Drehmoment bei max. Wirkungsgrad:	0,080 kg · cm
Strom bei max. Wirkungsgrad:	0,21 A
Ausgangsleistung bei max. Wirkungsgrad:	0,41 W

Pololu, [2024b](#), Specs

Der Roboter verfügt über zwei unabhängig arbeitende Motoren, von denen jeweils einer ein Rad antreibt. Es besteht keine mechanische Kopplung zwischen den beiden Motoren, was eine differenzierte Steuerung ermöglicht, etwa für Kurvenfahrten oder Drehungen auf der Stelle. Eine mechanische Kopplung würde den Einsatz eines größeren Motors sowie einer erheblich komplexeren Mechanik erfordern, einschließlich eines Differenzialgetriebes und einer zusätzlichen Lenkachse mit Servomotor. Diese Bauweise würde das Design erheblich aufwändiger machen. Die Entscheidung für zwei getrennte Motoren stellt daher die effizienteste Lösung für den Antrieb und die Steuerung des Roboters dar. Im Unterabschnitt 3.2 wird die Programmierung der Motoren genauer erläutert.

2.3 Sensorik des Roboters: Aufbau und Funktionsweise

Die Micromouse ist mit einer umfassenden Sensorik ausgestattet, die verschiedene Arten von Sensoren integriert. Da es sich bei der Micromouse um ein Lernprojekt handelt, bietet die Platine eine Vielzahl von Sensoren und Sensortypen, die gezielt getestet werden können. Diese Vielfalt unterstützt den Lernprozess der Anwender, indem sie verschiedene Ansätze zur Implementierung und Anwendung der Sensorik ermöglicht. Beispielsweise lassen sich für die Navigation verschiedene Ansätze und Methoden implementieren.

Diese vielseitige Sensorik prädestiniert die Micromouse nicht nur für ihren ursprünglichen Zweck, etwa als Demonstrationsplattform im Rahmen eines Maze-Solving-Projekts, sondern auch für eine Vielzahl weiterer Anwendungen im Bereich Robotik. Die spezifische Nutzung hängt von den individuellen Anforderungen und den Schwerpunkten des Projekts ab.

Die Micromouse umfasst die folgende Sensorik:

- **RC-Type QTR-Reflektionssensoren** (IR-Sensoren): Diese Sensoren werden primär für Line-Follow-Anwendungen und Stoßerkennung eingesetzt.
- **ST LSM6DSO**: Ein kombinierter Sensor mit 3-Achsen-Beschleunigungsmesser und 3-Achsen-Gyroskop zur Messung von Bewegung/Beschleunigung und Neigung.
- **ST LIS3MDL**: Ein Magnetsensor zur Erfassung von Magnetfeldern.
- **Time of Flight IR-Distanzsensor OPT3101**: Dieser Sensor spielt eine zentrale Rolle in der Distanzmessung und wird aufgrund seiner besonderen Bedeutung im Projekt ausführlich beschrieben.
- **Encoder**: Der Encoder erfasst die Radposition und gibt diese als Ganzzahlwert zurück, welcher typischerweise der Anzahl der erfassten Impulse entspricht. Dadurch lassen sich sowohl Position als auch Bewegungsrichtung des Rades bestimmen.

Linien- und Stoßsensoren

Der 3pi+ 2040 ist standardmäßig mit insgesamt 7 Sensoren ausgestattet, die zur Erkennung von Linien und Kollisionen dienen. Dabei handelt es sich um fünf nach unten gerichtete Liniensensoren sowie zwei nach vorne gerichtete Stoßsensoren.

Pinbelegung und Umbelegung:

Die Standardpinbelegung für die Linien- und Stoßsensoren auf der Platine des 3pi+ 2040 sind wie folgt:

- **GP26** steuert den Infrarotemitter der Liniensensoren (DNE).
- **GP22** bis **GP18** steuern die einzelnen Liniensensoren (DN1 bis DN5).
- **GP17** und **GP16** steuern die linken und rechten Stoßsensoren (BL, BR).
- **GP23** steuert den Infrarotemitter der Stoßsensoren (BE).

Pololu, [2024c](#), S. 6.5 Eine Übersicht über die Belegung und Funktionen der einzelnen GPIO Pins können in der GPIO Tabelle 7 im Anhang A nachgeschaut werden.

Falls nötig, können diese Anschlüsse umbelegt werden, indem man die Verbindungen auf der Unterseite der Platine durchtrennt und neue Verbindungen zu anderen GPIO-Pins herstellt. Davon wird jedoch abgeraten, da dieses Vorgehen irreversibel und sehr fehleranfällig ist da aktiv die Leiterkarte beschädigt wird.

Liniensensoren

Die fünf Liniensensoren befinden sich an der Unterseite des Roboters entlang der Vorderkante. Sie bestehen aus Infrarot-LEDs, die nach unten abstrahlen, und Phototransistoren, die das reflektierte Infrarotlicht messen. Pololu, [2024c](#), S. 6.5 Diese Sensoren helfen dem 3pi+ dabei, zwischen hellen und dunklen Oberflächen zu unterscheiden.

Die Funktionsweise dieser Sensoren basiert auf dem Prinzip einer RC-Schaltung und wird über die I/O-Pins des Mikrocontrollers gesteuert. Ein I/O-Pin des RP2040-Mikrocontrollers aktiviert den Sensor, indem er eine definierte Ausgangsspannung anlegt. Anschließend wird die Zeit gemessen, die der Kondensator benötigt, um sich zu entladen. Diese Entladezeit ist direkt von der Helligkeit der reflektierten Oberfläche abhängig.

Je intensiver das reflektierte Licht, desto stärker wird der Phototransistor leitend, wodurch der Kollektor-Emitter-Widerstand abnimmt. Ein niedrigerer Widerstand führt zu einer schnelleren Entladung des Kondensators. Umgekehrt verzögert ein höherer Widerstand die Entladung. Die Entladezeit kann anschließend über einen I/O-Pin des Mikrocontrollers ausgelesen werden und liefert eine Rückmeldung über die Lichtintensität der gemessenen Fläche. Der detaillierte Schaltplan ist im Anhang C unter Linesensors abgebildet.

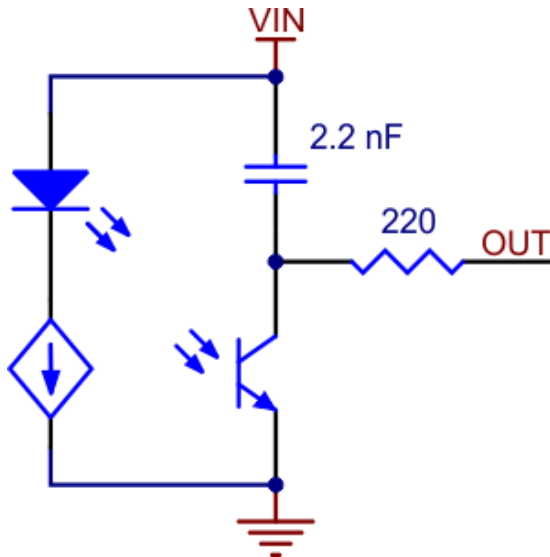


Abbildung 4: Schaltplan eines QTR-Sensorkanals in RC-Beschaltung mit dimmbarer Emitter-LED.

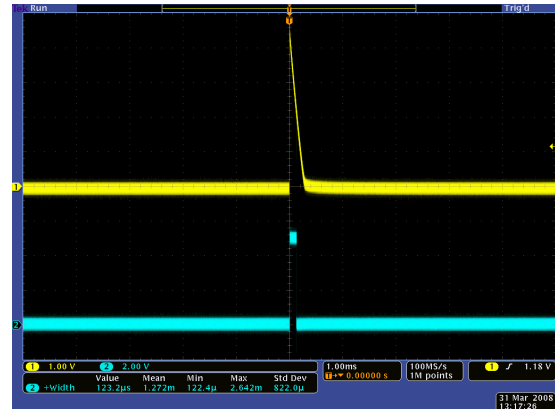


Abbildung 5: Ausgangssignal des QTR-RC-Sensors (gelb) und die zugehörige Zeitmessung des Mikrocontrollers (blau) bei einem Abstand von 1/8 Zoll über einer weißen Oberfläche.

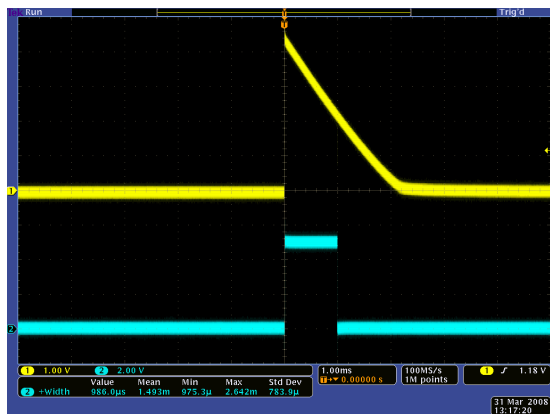


Abbildung 6: Ausgangssignal des QTR-1RC-Sensors (gelb) und die zugehörige Zeitmessung des Mikrocontrollers (blau) bei einem Abstand von 1/8 Zoll über einer weiß-schwarzen Grenzfläche.

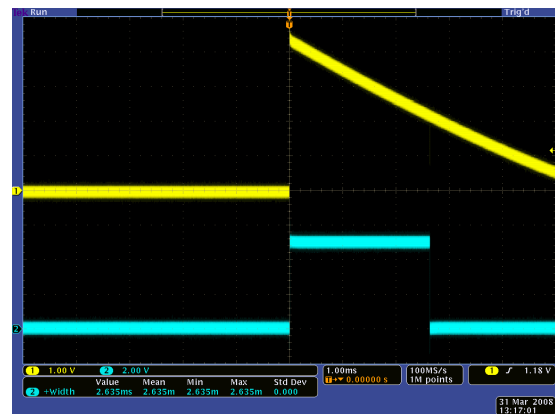


Abbildung 7: Ausgangssignal des QTR-1RC-Sensors (gelb) und die zugehörige Zeitmessung des Mikrocontrollers (blau) bei einem Abstand von 1/8 Zoll über einer schwarzen Linie.

Die Sensoren sind von links nach rechts von 1 bis 5 durchnummeriert, der linke Sensor entspricht also der 1. Im Schaltplan sind die Sensoren als **DOWN1** bis **DOWN5** bezeichnet. Auf der Platine sind sie an den dafür bereitgestellten Durchkontaktierungen als **DN1** bis **DN5** bezeichnet. Die IR-Emitter der Liniensensoren werden über das Signal **DOWNEMIT** (DNE) gesteuert. Diese werden zentral mit einem Befehl für alle Emitter gesteuert. Diese Vorgehensweise reduziert den Bedarf an Pins und verbessert die Übersichtlichkeit. Eine separate Ansteuerung der Emitter-LED ist nicht erforderlich, da sie ausschließlich der Bereitstellung von Infrarotlicht dient und nicht für eine Messung vorgesehen ist. Es ist lediglich erforderlich, dass die LEDs aktiviert sind. In 3.3 stehen weitere Informationen zur Programmierung und Anwendung der Sensoren. Pololu, 2024c, S. 6.5

Stoßsensoren

Die beiden Stoßsensoren dienen dazu, physische Kontakte mit Objekten vor dem Roboter zu erkennen. Diese Sensoren sind ebenfalls Reflexionssensoren, messen jedoch die Änderung der reflektierten IR-Lichtintensität, die durch das Eindringen der zugehörigen Plastikklappen, den sogenannten *Flaps*, an der Stoßseite des Roboters verursacht werden. Sie können bestimmen, auf welcher Seite des Roboters ein Hindernis erkannt wurde, basierend darauf, welcher Sensor auf Änderung reagiert. Im Schaltplan sind die Signale als **BUMPL** und **BUMPR** bezeichnet. Auf der Platine sind sie als **BL** und **BR** beschriftet. Die IR-Emitter der Stoßsensoren werden über das Signal **BUMPEMIT** (*BE*) gesteuert. Das Prinzip des Sensors ist das gleiche wie bei den Liniensensoren, und kann im vorherigen Abschnitt Unterabschnitt 2.3 nachgelesen werden.

Die Anordnung der Komponenten zur Stoßerkennung unterscheidet sich deutlich von der Positionierung der Liniensensorkomponenten. Die Emitter-LED ist oberhalb der Platine angebracht und richtet ihr Infrarotlicht gezielt auf die Stoßflügel, die sich links und rechts des Roboters befinden. Zwischen der Platine und den Stoßflügeln befindet sich ein schmaler Spalt, der es ermöglicht, dass das emittierte Infrarotlicht reflektiert und auf den darunterliegenden Phototransistor trifft.

Im Falle einer Kollision mit einem Hindernis werden die Stoßflügel in Richtung der Platine gedrückt, wodurch der zuvor vorhandene Spalt geschlossen wird. Infolgedessen wird weniger bis gar kein Licht mehr auf den Phototransistor reflektiert, was eine Kollision signalisiert.

Die Phototransistoren sind auf der Unterseite der Platine angebracht. Diese Positionierung dient dem Schutz vor externen Lichtquellen und minimiert potenzielle Interferenzen durch Umgebungslicht. Durch diese Anordnung wird sichergestellt, dass die Erkennungsmechanismen präziser arbeiten und ausschließlich auf die Lichtreflexionen der Emitter-LED reagieren.

Jeder Sensor ist mit einem Vorwiderstand von $220\,\Omega$ abgesichert, um den maximal möglich fließenden Strom zu begrenzen und die Bauteile vor Schäden durch Überstrom zu schützen. Eine wichtige technische Herausforderung ergibt sich aus der gegenseitigen Beeinflussung der Sensoren: Die Infrarot-Emitter der Liniensensoren können die Messungen der Stoßsensoren stören und umgekehrt. Um präzise Messergebnisse zu gewährleisten, ist es daher ratsam, entweder nur die Liniensensoren oder nur die Stoßsensoren gleichzeitig zu aktivieren. Pololu, 2024c, S. 6.5

Die Intensität der Emitter-LEDs kann durch Dimmung reguliert werden, was die Anpassung an unterschiedliche Umgebungslichtverhältnisse ermöglicht. Eine unzureichende oder übermäßige Einstellung der Infrarotlichtstärke hat jedoch negative Auswirkungen

auf die Messergebnisse. Ist das ausgestrahlte Licht zu stark, kann der Sensor in die Sättigung gehen, wodurch keine brauchbaren Daten mehr gemessen werden können. Bei zu geringer Lichtintensität hingegen ist das reflektierte Signal möglicherweise zu schwach, um zuverlässig interpretiert zu werden. Für die Unterscheidung von nur zwei Zuständen ist die richtige Lichtkalibrierung eher nebensächlich. Je genauer die Grauwerte gemessen werden sollen, desto wichtiger wird die richtige Lichtkalibrierung des Sensors, beispielsweise beim Liniensensor.

Da beide Sensortypen auf der Reflexion von Infrarotlicht basieren, sind sie besonders anfällig für Störungen durch externe Lichtquellen, insbesondere Sonnenlicht oder künstliche Beleuchtung mit hohem Infrarotanteil. Eine Kalibrierung der Sensoren zu Beginn der Nutzung ist daher essenziell, um die Auswirkungen solcher Störfaktoren zu minimieren. Für die Liniensensoren kann dies durch eine Vorabmessung der Reflexion von hellen und dunklen Oberflächen erfolgen, die als Referenzwerte für die Interpretation späterer Messungen dienen. In Kapitel 3.3 stehen weiter Informationen zur Anwendung und Programmierung der Stoßsensoren.

Die Inertiale Messeinheit (IMU): Aufbau, Funktionsweise und Kalibrierung

In diesem Kapitel beschränke ich mich auf die für die Studienarbeit relevanten Aspekte der Sensorik, da eine umfassendere Darstellung den Rahmen dieser Arbeit sprengen würde. Die nachfolgenden Abschnitte behandeln die Trägheitssensorik sowie die Magnetfeldsensorik. Beide Sensoren kommunizieren über den I²C-Bus mit dem Mikrocontroller. Pololu, [2024c](#), S. 6.6

Weiterführende Informationen zur Programmierung und Ansteuerung der IMU sind in Kapitel 3.3 zu finden.

Always-on-3D-Beschleunigungsmeter und 3D-Gyroskop

Auf der Platine des Roboters befindet sich der Chip ST LSM6DSO, welcher ein 3D-Digitalbeschleunigungsmesser und ein 3D-Digitalgyroskop kombiniert. Somit liefert der Sensor Werte über 6D, also 3 Achsen bezüglich Beschleunigung und 3 Achsen bezüglich der Drehung im Raum. Der Beschleunigungssensor kann für folgende Bereiche eingestellt werden: $\pm 2\text{ g}$, $\pm 4\text{ g}$, $\pm 8\text{ g}$, $\pm 16\text{ g}$. Die Einheit bezieht sich in diesem Fall auf die Erdbeschleunigung von $9,81\text{ m/s}^2$ wobei dies 1 g entspricht. Das digitale Gyroskop kann für folgende Bereiche eingestellt werden: $\pm 125^\circ/\text{s}$, $\pm 250^\circ/\text{s}$, $\pm 500^\circ/\text{s}$, $\pm 1000^\circ/\text{s}$, $\pm 2000^\circ/\text{s}$ dps (degrees per second). Für eine Messung kann immer nur ein Bereich ausgewählt werden. Größere Bereiche ermöglichen die Erfassung größerer Beschleunigungen oder Rotationen, sind aber bei kleineren Beschleunigungen weniger genau. Kleinere Messbereiche bieten eine höhere Genauigkeit bei kleinen Beschleunigungen, können aber große Beschleunigungen nicht vollständig erfassen und nehmen den Maximalwert an. Dies ist darauf zurückzuführen, dass sich die Toleranz prozentual auf den Messwert auswirkt. Daher sollte der Messbereich entsprechend zu den erwarteten Messwerten ausgewählt werden. Wenn man sich nicht sicher ist, kann man einen hohen Bereich wählen, sich die Messwerte ausgeben lassen und dann den Bereich herunterskalieren. STMicroelectronics, [2019](#), S. 1f. Description

Die lineare Beschleunigungsempfindlichkeit eines Sensors lässt sich durch die Applikation einer Beschleunigung von 1 g ermitteln. Da der Sensor in der Lage ist, Gleichstrom-Beschleunigungen (DC-Beschleunigungen) zu messen, kann dies einfach durchgeführt werden, indem die zu prüfende Achse des Sensors in Richtung des Erdbodens ausgerichtet und der Ausgangswert aufgezeichnet wird.

Die Nutzung der Erdbeschleunigung bietet sich an, da diese als konstante Größe über einen längeren Zeitraum hinweg unverändert bleibt. Dies unterscheidet sich von Wechselstrom-Beschleunigungen (AC-Beschleunigungen), die durch zeitlich variable Bewegungen wie Vibrationen oder Schwingungen entstehen. Die Erdbeschleunigung stellt eine universell verfügbare und leicht einsetzbare Referenz dar. Eine Kalibrierung ist unkompliziert durchführbar, indem die zu messende Achse des Sensors lediglich in Richtung des Erdbodens ausgerichtet wird.

Um die Empfindlichkeit genauer zu bestimmen, wird der Sensor anschließend um 180° gedreht, sodass die Achse in Richtung des Himmels zeigt, und der Ausgangswert erneut gemessen wird. Der Sensor erfährt hierbei eine Beschleunigung von $\pm 1\text{ g}$. Die tatsächliche Empfindlichkeit ergibt sich, indem der größere Ausgangswert vom kleineren subtrahiert und das Ergebnis durch 2 geteilt wird. Dieser Wert zeigt im Lauf der Zeit nur minimale Veränderungen durch Temperaturschwankungen oder Alterung.

Das Gyroskop liefert hingegen ein positiv ansteigendes digitales Ausgangssignal, wenn eine Drehbewegung gegen den Uhrzeigersinn um die jeweilige Achse erfolgt. Die Empfindlichkeit eines Gyroskops beschreibt den Verstärkungsfaktor und kann durch das Anlegen einer definierten Winkelgeschwindigkeit bestimmt werden. Dieser Wert bleibt ebenfalls weitgehend stabil, auch unter Einfluss von Temperatur- oder Alterungseffekten. In der Praxis gestaltet sich die Realisierung einer gleichmäßigen Winkelgeschwindigkeit jedoch aufwändig, weshalb eine gewisse Messungenauigkeit bei der Projektierung berücksichtigt werden sollte. STMicroelectronics, 2019, S. 16

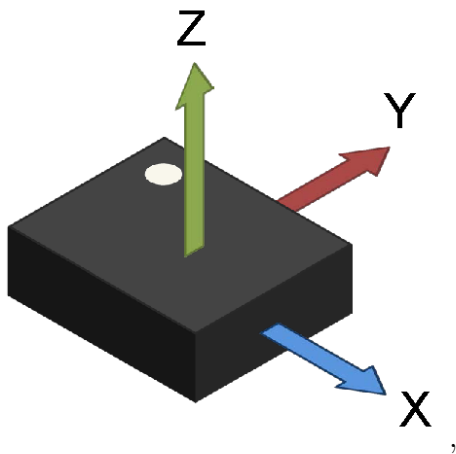


Abbildung 8: Richtung der feststellbaren Beschleunigung
STMicroelectronics, 2019, S. 6

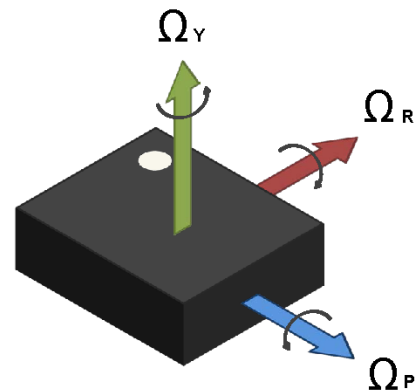


Abbildung 9: Richtung der feststellbaren Drehrate
STMicroelectronics, 2019, S. 6

Die Sensorik des Roboters ermöglicht es, Fehler zu erkennen, die durch äußere Einwirkungen entstehen und das erwartete Verhalten beeinträchtigen. Ein Feedback-System, das auf den Sensordaten basiert, kann Abweichungen zwischen der geplanten und der tatsächlichen Ausführung von Aktionen identifizieren und entsprechende Gegenmaßnahmen einleiten. Ein typisches Beispiel ist die Erkennung von Kollisionen: Trifft der Roboter auf ein Hindernis, wird eine abrupte und deutliche negative Beschleunigung erzeugt, die vom integrierten Beschleunigungssensor erfasst wird, sodass Gegenmaßnahmen ergriffen werden können. Auf diese Weise kann einer potenziellen Schadensentstehung vorgebeugt werden. Die Sensordaten des Gyroskops können für Drehungen um die eigene Achse genutzt werden um die Kurvennavigation zu verbessern.

Magnetometer

Auf der Platine des Roboters befindet sich außerdem der Magnetsensor **ST LIS3MDL**, auch 3-Achs-Magnetometer genannt. Pololu, [2024c](#) Der Magnetsensor misst Magnetfelder in den Dimensionen X,Y und Z. Es kann hier, wie auch beim Trägheitssensor, ein Wertebereich eingestellt werden: $\pm 4 \pm 8 \pm 12 \pm 16$ Gauss. STMicroelectronics, [2015](#), S. 1 Gauss ist die Einheit der magnetischen Flussdichte und entspricht $1 \text{ Gauss (G)} = 10^{-4} \text{ Tesla (T)}$ Wasserman, [2021](#). Dabei ist zu beachten, dass ein größerer Messbereich auf Kosten der Auflösung ein stärkeres Magnetfeld messen kann. Die Messbereichsauswahl verhält sich ähnlich wie bei dem Trägheitssensor in Abbildung 2.3 und kann genauso durchgeführt werden.

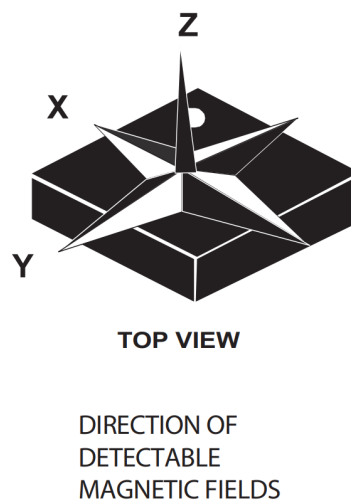


Abbildung 10: Magnetsensorachsen
STMicroelectronics, [2015](#), S. 6

OPT3101 - Time-of-Flight (ToF)

Bei dem verwendeten Sensor handelt es sich um ein 3-Kanal Time-of-Flight (ToF) Distanz- und Näherungssensormodul, das auf dem OPT3101-IC von Texas Instruments basiert. Das Modul nutzt Infrarotlicht (Wellenlänge: 940 nm), das gepulst mit einer Frequenz von 10 MHz ausgesendet wird. Die Distanzmessung erfolgt nicht – wie bei herkömmlichen IR-Sensoren – über die Lichtintensität, sondern durch die Phasenverschiebung des reflektierten Signals zu dem ausgesendetem Signal, was eine genauere Bestimmung der Entfernung zum Zielobjekt ermöglicht.

Das Modul ist mit drei unabhängig wählbaren Kanälen ausgestattet, die jeweils durch zwei LEDs realisiert werden. Diese drei Kanäle decken insgesamt ein Sichtfeld (Field-of-View, FOV) von 160° ab. Der linke Kanal TX0 50° , der mittlere Kanal TX1 60° , der rechte TX2 50° . Die maximale Reichweite des Sensors liegt – abhängig von Objekt und Umgebungsbedingungen – bei etwa 1 m. Pololu, [2024a](#)

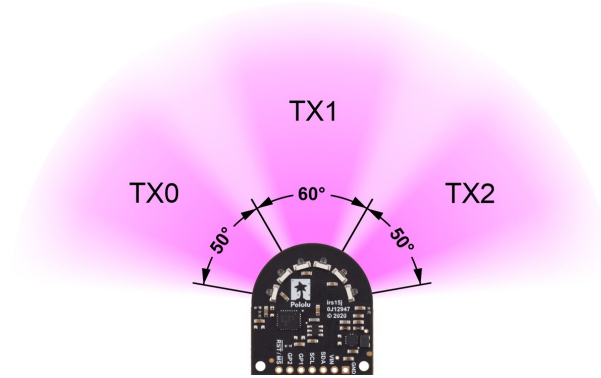


Abbildung 11: OPT3101 - Channel FOV
Pololu, [2024a](#)

Die Distanzmessdaten können über eine digitale I²C-Schnittstelle ausgelesen werden, die auch zur Konfiguration des Sensors dient. Die Kommunikation erfolgt über die üblichen I²C-Leitungen SCL (Takt) und SDA (Daten). Dank integrierter Level-Shifter ist die I²C-Schnittstelle kompatibel mit Logikpegeln von 3,3 V bis 5 V, was eine einfache Integration in verschiedene Systeme ermöglicht. Pololu, [2024a](#)

Der Sensor verfügt zudem über einen internen Spannungsregler, der aus einer Eingangsspannung von 2,5 V bis 5,5 V die notwendige Betriebsspannung von 3,3 V für den OPT3101 bereitstellt. Jedes Modul enthält einen werksseitig kalibrierten EEPROM mit 256 Byte, in dem individuelle Kalibrierdaten gespeichert sind. Diese Kalibrierung wird beim Start automatisch vom OPT3101 geladen und erlaubt eine präzise Distanzmessung ab Werk. Eine erneute interne Crosstalk-Kalibrierung ist in der Regel nicht notwendig, da diese bereits in der Werkskalibrierung berücksichtigt wurde. Pololu, [2024a](#)

Die Konfiguration des Sensors erfolgt ebenfalls über die I²C-Schnittstelle. Durch die Kanalwahl (TX0, TX1, TX2) wird die Richtung bestimmt, in die das Modul Infrarotlicht aussendet. Außerdem kann die Helligkeit der LEDs bestimmt werden, die Sample-Rate und die Anzahl der Subframes. Eine Messung besteht aus mehreren Subframes, die gemittelt den Messwert ergeben. Pololu, [2024a](#) Weitere Informationen zur Programmierung stehen im Kapitel 3.3. Weiter Informationen zum Sensor selbst sind dem Anhang D zu entnehmen, bzw. dem beigefügtem Datenblatt. Der Schaltplan des Distanzsensors ist in Anhang E zu finden.

2.4 OLED-Display (SH1106)

Zur direkten Ausgabe von Informationen und zur Erstellung einer Benutzeroberfläche ist der Roboter mit einem grafischen OLED-Display ausgestattet. Das monochrome (weiße) Display basiert auf dem SH1106-Controller und bietet eine Auflösung von 128×64 Pixeln. Angeschlossen wird das Modul über eine 4-Draht-SPI-Schnittstelle (Serial Peripheral Interface), die eine schnelle Datenübertragung ermöglicht.

Dank eines integrierten Spannungsreglers kann das Display mit einer Versorgungsspannung zwischen 3,1 V und 5,5 V betrieben werden, wobei die Logikpins für die Datenübertragung auf 3,3 V ausgelegt sind. Die wesentlichen technischen Daten sind in Tabelle 3 zusammengefasst, die Pinbelegung in Tabelle 4.

Tabelle 3: Technische Spezifikationen des SH1106 OLED-Displays

Eigenschaft	Wert
Controller	SH1106
Auflösung	128 × 64 Pixel
Aktive Anzeigefläche	29,4 mm × 14,7 mm (1,3 Zoll Diagonale)
Abmessungen	35,4 mm × 33,5 mm
Versorgungsspannung (VCC)	3,1 V bis 5,5 V
Logikspannung	3,3 V
Stromaufnahme	ca. 40 mA (typisch, alle Pixel an)

Tabelle 4: Pinbelegung des OLED-Displays

Pin	Beschreibung
GND	Masse (0 V)
VCC	Versorgungsspannung
CLK	SPI Clock (Taktsignal)
MOS	SPI Data (MOSI - Master Out Slave In)
RES	Reset-Leitung
DC	Data/Command Select (Daten-/Befehlsauswahl)
CS	SPI Chip Select

Pololu, [2025a](#) Weitere Informationen zur Ansteuerung des Displays stehen in Unterabschnitt 3.4. Das Datenblatt ist im Anhang F bzw. digitalen Anhang.

2.5 Kommunikationsschnittstellen und Pinbelegung

Überblick über die Funktion von I²C

Das Inter-Integrated Circuit (I²C)-Protokoll ist ein weit verbreitetes serielles Kommunikationsprotokoll, das von über 50 IC-Herstellern in Komponenten angeboten wird. Es dient zur einfachen und effizienten Kommunikation zwischen Mikrocontrollern und Peripheriegeräten wie Sensoren, Displays, Speichern oder Aktoren. Pini, [2020](#) Der I²C ist ein bidirektionaler, serieller Kommunikationsbus, der im Master-Slave-Prinzip arbeitet. Das bedeutet, dass ein oder mehrere Master-Geräte, häufig Mikrocontroller, die Kommunikation initiieren und steuern. Slave-Geräte senden ihre Daten erst, wenn sie vom Master adressiert werden. Valdez und Becker, [2015](#), S.6 Für die Kommunikation werden zwei Leitungen verwendet, Serial Data (SDA), der die Daten überträgt, und Serial Clock (SCL), der den Takt überträgt. Mankar et al., [2014](#), S.2. Beide Leitungen sind über Pull-Up-Widerstände an die Versorgungsspannung (VCC) angeschlossen, sodass sie im Leerlauf (Bus idle) im High-Zustand sind. Der Datentransfer wird auch nur initiiert, wenn der Bus im Idle-Zustand ist. Valdez und Becker, [2015](#), S.3. Die Datenübertragung erfolgt in Form von Bytes, also jeweils 8 Bits, wobei das Most Significant Bit (MSB) zuerst gesendet wird.

Der Ablauf für das Schreiben auf den I²C-Bus ist folgendermaßen:

- **Start-Bedingung:**
Der Master setzt den Bus in den Kommunikationsmodus, indem er eine Startbedingung sendet. Dies geschieht, wenn SDA von High auf Low wechselt, während SCL High ist. Valdez und Becker, 2015, S.4 Figure 5.
- **Adressierung und R/W-Bit:**
Direkt nach dem Start sendet der Master die Adresse des Ziel-Slave-Geräts, gefolgt von einem R/W-Bit (Read/Write). Dabei gilt wenn R/W = 0 ist, handelt es sich um einen Schreibvorgang. Wenn R/W = 1 ist handelt es sich um einen Lesevorgang.
- **Acknowledge (ACK):**
Jedes gesendete Byte, ob Adresse, Register oder Daten, wird vom Empfänger mit einem ACK-Bit (Acknowledge) bestätigt. Dieses Bit zeigt an, dass das Byte korrekt empfangen wurde. Valdez und Becker, 2015, S.5
- **STOP-Bedingung:**
Die Übertragung wird beendet, indem der Master eine Stop-Bedingung sendet. Das geschieht, wenn SDA von Low auf High wechselt, während SCL High ist. Valdez und Becker, 2015, S.4 Figure 5.
- **Repeated Start:**
Bei Bedarf kann der Master auch einen sogenannten Repeated Start senden, um eine neue Kommunikation zu beginnen, ohne den Bus zunächst freizugeben. Das geschieht noch vor der Stop-Bedingung wenn der Bus noch nicht im Grundzustand (idle) ist. Das ist besonders in Multi-Master-Umgebungen nützlich, wenn der Master den Bus noch nicht freigeben will.

Die Verwendung von I²C bietet verschiedene Vorteile: Der Verkabelungsaufwand ist sehr gering, da nur zwei Leitungen für den Bus notwendig sind. Das reduziert Platz auf der Platine, und reduziert damit auch die Komplexität des Leiterkartenlayouts. Es können außerdem bis zu 128 Geräte an einen Bus angeschlossen werden, da jedes Gerät über eine eindeutige Adresse verfügt. FMH-Studios, 2020 I²C ist sehr leicht zu implementieren, da es schon viele fertige Bibliotheken dafür gibt und weit verbreitet ist. Durch den weiten Gebrauch basieren viele Technologien auf diesen Bus, wodurch dazu viel Dokumentation und Software zur Verfügung steht. Der Bus ist flexibel einsetzbar, von einfachen Sensorverbindungen bis hin zu komplexen Systemen mit mehreren Peripheriegeräten.

Da der I²C-Bus in der Micromouse verwendet wird, erscheint es sinnvoll, im Rahmen dieser Arbeit ein eigenes Kapitel diesem Kommunikationsstandard zu widmen. Ziel ist es, ein grundlegendes Verständnis für die Funktionsweise und Anwendungsmöglichkeiten des I²C-Busses zu vermitteln. Für die Kommunikation über I²C stehen umfangreiche Softwarebibliotheken im Internet zur Verfügung. Es ist daher in der Praxis nicht notwendig, ein eigenes Kommunikationsprotokoll zu entwickeln. Dennoch ist eine direkte Implementierung, aus Interesse oder zu Lernzwecken, des Busses mit der Micromouse möglich. Für produktive Anwendungen wird jedoch klar zur Nutzung vorhandener Bibliotheken geraten.

3 Programmierung und Inbetriebnahme

3.1 Einrichtung der Entwicklungsumgebung

Zu Beginn der Inbetriebnahme ein kleines Vorwort. Wie zuvor schon beschrieben, kann der Roboter auf verschiedene Arten programmiert werden. Zur Programmierung stehen mehrere Programmiersprachen zur Auswahl, die gängigsten sind hierbei wohl C, C++ und Python. Die Programmierumgebung kann selbstverständlich frei gewählt werden, es sollte darauf geachtet werden, dass der Editor den Mikrocomputer auch programmieren kann, und eine Schnittstelle für die Beschreibung des Chips bereitsteht. Im Rahmen der Studienarbeit wird hierfür die Arduino-IDE verwendet und als Beispiel einer erfolgreichen Inbetriebnahme hergezogen. Eine weitere kostenlose (Stand 2025) Programmierumgebung ist Visual-Studio-Code von Microsoft mit dem Platform-IO-Plug-In.

Der Roboter Pololu 3Pi Plus 2040 basiert auf einem Raspberry-Pi-Pico-Mikrocontroller (RP2040) und bietet flexible Programmiermöglichkeiten in Sprachen wie C, C++, MicroPython und Arduino C++. Die Pinbelegung sowie detaillierte Informationen zu den Anschlussmöglichkeiten und Spezifikationen sind dem Datenblatt im Anhang G zu entnehmen. Die GPIO-Pins können individuell angesteuert werden. Eine detaillierte Übersicht der Pinbelegung befindet sich im Anhang A in der Tabelle 7. Dort steht, welcher GPIO-PIN welcher Hardware zugeordnet ist, und welche noch frei verfügbar sind. Es empfiehlt sich jedoch, nur ungenutzte Pins für die eigene Programmierung zu verwenden, um Signalstörungen und Interferenzen zu vermeiden. Zur Erweiterung der Hardware bietet der Roboter eine I²C-Schnittstelle, über die sich zusätzliche Sensoren oder Module anschließen lassen, ohne die Anzahl der verfügbaren Pins zu reduzieren. Als Programmierschnittstelle dient ein USB-C-Anschluss, über den sich der Roboter mit einem Computer verbinden lässt. Der Roboter wird standardmäßig mit einem vorinstallierten Testprogramm in MicroPython ausgeliefert, das grundlegende Funktionen wie die Sensorprüfung unterstützt und sicherstellt, dass alle integrierten Komponenten ordnungsgemäß funktionieren. Hierbei ist zu beachten, dass die Motoren sich nur drehen, wenn auch Batterien eingelegt sind. Wird der Roboter nur über die USB-C-Schnittstelle mit Strom versorgt, werden sich die Motoren nicht drehen. Für eine Programmierung in einer anderen Sprache, beispielsweise in C++, muss der Roboter zunächst in den Bootloader-Modus versetzt werden, um die MicroPython Firmware zu überschreiben. Dazu drückt man vor dem Anschließen der Stromversorgung die „B“-Taste und stellt dann erst die Spannungsversorgung her, oder nutzt alternativ die Resettaste. In Abbildung 12 ist der Taster markiert mit dem man den Mikrocontroller in den Bootloader setzen kann. Zur Erklärung: die Resettaste verhält sich wie eine kurzzeitige Stromunterbrechung. Falls der Roboter mit dem Standardprogramm programmiert ist, kann der Bootloader auch über das Menü aufgerufen werden. Dieser Modus ermöglicht das direkte Aufspielen eines neuen Programms und ermöglicht den Wechsel zwischen verschiedenen Programmiersprachen. Für die einfache Suche ist hier der direkte Link zur [GitHub Pololu 3pi plus 2040 Bibliothek](#) Green, 2024 Im Anhang H sind weitere Links zu nützlichen Bibliotheken aufgeführt.

Es sind bereits benutzerfreundliche Bibliotheken auf der Plattform GitHub verfügbar, die die Ansteuerung der Sensoren und anderer Komponenten des Roboters vereinfachen und beschleunigen. Diese Bibliotheken sind speziell auf den Pololu 3pi Plus 2040 abgestimmt und bieten Funktionen für die Sensorabfrage und Motorsteuerungen sowie den generellen Zugriff auf den Datenbus. Informationen zu der Bibliothek und wie man sie einbindet steht in 3.1 und in den einzelnen Unterkapiteln zu der Programmierung der Sensoren.

Die nachfolgende Abbildung zeigt die Position der in diesem Abschnitt beschriebenen Knöpfe.



Abbildung 12: Pololu 3pi+ 2040 B- und Resetbutton

Weitere Links für das Programmieren mit Python und C stehen im Anhang J und Anhang K.

Installation und Konfiguration der Arduino-IDE

Die Arduino-IDE (Integrated Development Environment) ist eine kostenlose Entwicklungsumgebung, die über die offizielle [Arduino-Website](#) bezogen werden kann. Im Download-Bereich der Website wird das passende Installationspaket für das jeweilige Betriebssystem (Windows, macOS oder Linux) ausgewählt und die Installation durchgeführt.

Nach dem ersten Start der IDE muss die Board-Unterstützung für den Pololu 3pi+ 2040 Roboter hinzugefügt werden. Obwohl viele gängige Arduino-Boards vorinstalliert sind, erfordert der RP2040-Mikrocontroller ein zusätzliches Paket. Die Installation erfolgt über den Menüpunkt **Werkzeuge** → **Board** → **Boardverwalter...**. Im sich öffnenden Fenster wird nach „Arduino Mbed OS RP2040 Boards“ gesucht und das entsprechende Paket installiert.

Zur Veranschaulichung zeigt Abbildung 13 den Navigationspfad zur Auswahl des Boards nach erfolgreicher Installation des Pakets.

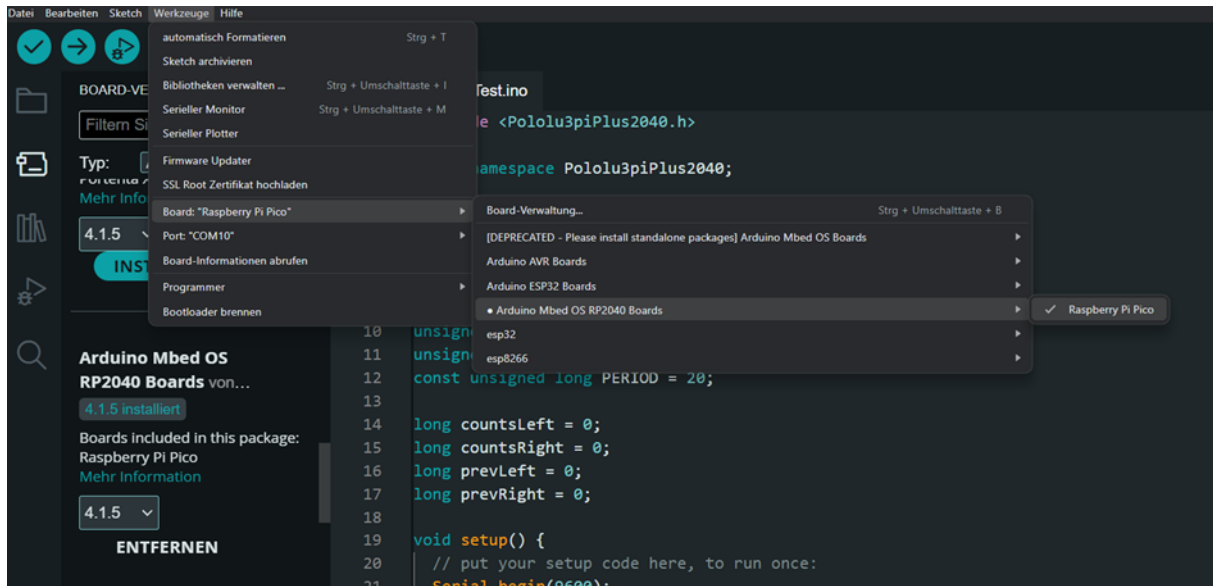


Abbildung 13: Pfad zur Boardauswahl in der Arduino-IDE

Jeder Arduino-Sketch, wie ein Programm in der Arduino-Umgebung genannt wird, basiert auf zwei Hauptfunktionen: `setup()` und `loop()`.

- **`setup()`:** Diese Funktion wird genau einmal ausgeführt, wenn der Mikrocontroller startet – also direkt nach dem Hochladen eines neuen Sketches oder nach einem Reset. Sie wird für Initialisierungen verwendet, wie zum Beispiel die Konfiguration von Pins als Ein- oder Ausgänge.
- **`loop()`:** Nach dem einmaligen Durchlauf der `setup()`-Funktion wird der Code innerhalb der `loop()`-Funktion kontinuierlich in einer Endlosschleife ausgeführt. Hier befindet sich das eigentliche Hauptprogramm.

Um die korrekte Konfiguration der Entwicklungsumgebung und die Kommunikation mit dem Roboter zu überprüfen, wird ein erster Funktionstest empfohlen. Ziel ist es, eine auf der Platine des Roboters integrierte LED zum Blinken zu bringen. Diese LED ist mit dem GPIO-Pin 25 verbunden, der in der Arduino-Umgebung über die Konstante `LED_BUILTIN` angesprochen werden kann. Der Code für diesen Test ist in Listing 1 dargestellt.

```

1  // Die setup()-Funktion wird einmal beim Start ausgeführt.
2  void setup() {
3      // Initialisiert den Pin der Onboard-LED (GPIO 25) als Ausgang.
4      pinMode(LED_BUILTIN, OUTPUT);
5  }
6
7  // Die loop()-Funktion wird kontinuierlich wiederholt.
8  void loop() {
9      digitalWrite(LED_BUILTIN, HIGH);    // Schaltet die LED ein.
10     delay(1000);                        // Wartet eine Sekunde (1000 ms).
11     digitalWrite(LED_BUILTIN, LOW);     // Schaltet die LED aus.
12     delay(1000);                        // Wartet eine weitere Sekunde.
13 }
```

Listing 1: Einfacher Blink-Sketch zur Funktionsüberprüfung

Vor dem Hochladen des Sketches muss sichergestellt werden, dass unter **Werkzeuge** das korrekte Board (z.B. „Raspberry Pi Pico“) und der richtige Port ausgewählt sind. Nachdem der Roboter über USB mit dem Computer verbunden wurde, erscheint in der Regel automatisch ein neuer COM-Port. Sollte dies nicht der Fall sein, kann der korrekte Port über den Gerätemanager des Betriebssystems identifiziert werden. Es ist zudem die Vorgehensweise aus Kapitel Unterabschnitt 3.1 zu beachten, falls der Roboter sich im Auslieferungszustand befindet und noch nicht für die Programmierung vorbereitet wurde.

Das Hochladen des Codes wird durch einen Klick auf das Pfeilsymbol in der oberen linken Ecke der IDE gestartet (siehe auch Abbildung 13). Wenn die LED auf der Platine anschließend im Sekundentakt blinkt, war der Prozess erfolgreich. Damit ist verifiziert, dass die gesamte Werkzeugkette – von der Codeerstellung über die Kompilierung bis zur Übertragung auf den Mikrocontroller – korrekt funktioniert.

Bibliothek einbinden

Die benötigte Bibliothek kann über die Plattform GitHub heruntergeladen werden: [Pololu 3pi+ Bibliothek](#). Green, 2024 Dazu klickt man auf der Repository-Seite auf den Button „Code“, woraufhin ein Dropdown-Menü erscheint. Dort wählt man „Download ZIP“, um die Bibliothek als ZIP-Datei herunterzuladen. Abbildung 14 veranschaulicht diesen Schritt.

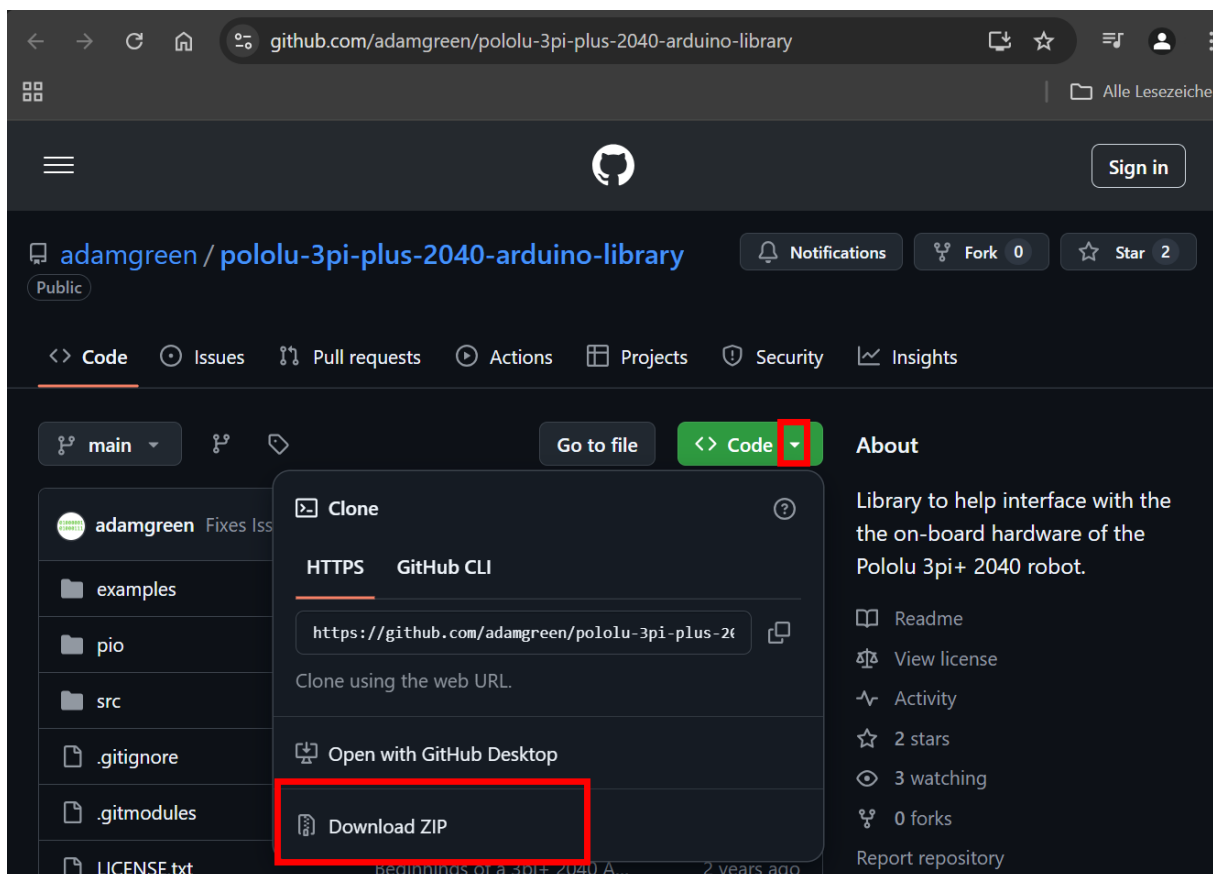


Abbildung 14: Bibliothek als ZIP-Datei herunterladen
Green, 2024

Anschließend muss die heruntergeladene Bibliothek in die Arduino-IDE eingebunden werden. Dazu wählt man in der Menüleiste „Sketch“ → „Bibliothek einbinden“ → „ZIP-Bibliothek hinzufügen“ und gibt den Pfad zur heruntergeladenen ZIP-Datei an. Nach erfolgreicher Einbindung kann die Bibliothek im Menü „Sketch“ → „Bibliothek einbinden“ überprüft werden – sie sollte dort in der Liste der verfügbaren Bibliotheken erscheinen.

Um die Bibliothek im eigenen Code zu verwenden, muss am Anfang des Programms folgender Include-Befehl eingefügt werden: `#include <Pololu3piPlus2040.h>`

3.2 Programmierung der Aktoren

Anwendung und Programmierung der Motoren

Nun betrachten wir die programmtechnische Seite, wie man die Motoren ansteuern kann. Um die Geschwindigkeit flexibel anpassen zu können, wird empfohlen, diese als Variable zu definieren. Dadurch lassen sich Änderungen leichter vornehmen, ohne den Code mehrfach anpassen zu müssen. Zur Steuerung der Motoren stehen zwei Methoden zur Verfügung. Die erste Variante besteht darin, eine einheitliche Geschwindigkeit für beide Motoren zu setzen. Dies geschieht durch die folgende Befehlsfolge:

```
1  Motors motors;
2
3  Velocity = 50;
4  motors.setLeftSpeed(Velocity);
5  motors.setRightSpeed(Velocity);
```

Listing 2: Einzelansteuerung der Motoren

Alternativ kann die Geschwindigkeit für den linken und rechten Motor separat definiert und mit einem einzigen Funktionsaufruf übergeben werden:

```
1  Motors motors;
2
3  VelocityLeft = 50;
4  VelocityRight = 50;
5  motors.setSpeeds(VelocityLeft, VelocityRight);
```

Listing 3: Einzelansteuerung der Motoren

Beide Methoden sind funktional gleichwertig, wobei die zweite Variante vor allem dann nützlich ist, wenn unterschiedliche Geschwindigkeiten für die beiden Motoren erforderlich sind, beispielsweise für Kurvenfahrten. Green, [2024](#)

Ansteuerung der LEDs

Auf der Platine des Roboters sind sechs RGB-LEDs verbaut: drei vorne und drei hinten. Zusätzlich befindet sich eine gelbe LED neben dem USB-C-Anschluss. Die RGB-LEDs können einzeln angesteuert werden, während die gelbe LED den GPIO-Pin 25 mit dem Taster A teilt. Pololu, [2024c](#), S. 6.2

Die Funktion `set(LED, RGB, Helligkeit)` ermöglicht die individuelle Steuerung jeder LED. Dabei kann jeder LED eine eigene Farbe sowie eine Helligkeit zugewiesen werden. Die sechs RGB-LEDs sind unter folgenden Bezeichnungen ansprechbar: `BACK_LEFT_LED`, `BACK_CENTER_LED`, `BACK_RIGHT_LED`, `FRONT_RIGHT_LED`, `FRONT_CENTER_LED`, `FRONT_LEFT_LED`. Green, [2024](#)

Jede Farbe wird durch einen RGB-Wert definiert, wobei die Intensität pro Farbkanal im Bereich von 0 bis 255 liegt. Die Mischung der Farben erfolgt durch entsprechende Kombination der RGB-Werte. Zudem gibt es vordefinierte Farben:

- BLACK (0, 0, 0)
- WHITE (255, 255, 255)
- RED (255, 0, 0)
- ORANGE (255, 128, 0)
- YELLOW (255, 255, 0)
- GREEN (0, 255, 0)
- BLUE (0, 0, 255)
- VIOLET (255, 0, 255)

Die Helligkeit kann separat eingestellt werden und liegt im Bereich von 0 bis 31. Dabei entspricht 0 aus und 31 der maximalen Helligkeit. Es ist zu beachten, dass eine höhere Helligkeit zu einem höheren Stromverbrauch führt und somit die Akkulaufzeit reduziert. Green, 2024

Zusätzlich zur individuellen Steuerung können alle LEDs gleichzeitig auf eine gemeinsame Helligkeit oder Farbe gesetzt werden:

```
1 leds.setBrightness(20); // Setzt alle LEDs auf Helligkeit 20
2 leds.set(RED(255, 0, 0)); // Alle LEDs leuchten rot
```

Listing 4: Ansteuerung der LEDs

Die gelbe LED muss zunächst initialisiert werden:

```
1 YellowLED yellowLED;
```

Listing 5: Initialisierung der gelben LED

Green, 2024

Nach der Initialisierung kann die integrierte LED angesteuert werden. Sie wird häufig für einfache Blinktests verwendet, da sie bereits auf der Platine verbaut ist und auch auf vielen anderen Entwicklungsboards, wie dem ESP32 oder klassischen Arduino-Boards, standardmäßig vorhanden ist. Diese LED dient in der Praxis häufig dazu, zu überprüfen, ob der Code erfolgreich hochgeladen wurde, bevor komplexere Programme ausgeführt werden.

Die LEDs können auch als visuelles Feedbacksystem für den Roboter genutzt werden, indem sie Statusanzeigen oder Reaktionen auf Eingaben von Sensoren oder Tastern darstellen. So könnte beispielsweise die Position des Roboters über einer Linie visualisiert werden, indem drei LEDs entsprechend der Sensorrückmeldungen gedimmt oder erhellt werden.

Buzzer ansteuern

Auf der Leiterkarte des Roboters befindet sich ein Piezo Buzzer mit dem einfache Töne generiert werden können. Dieser ist standardmäßig an Pin GP7 angeschlossen, welcher als PWM3-B konfiguriert werden kann. Wie ein PWM-Signal aufgebaut ist, wurde in Unterabschnitt 2.2 erläutert. Pololu, 2024c, S. 6.2 Sobald man den Pin in einer bestimmten Frequenz High und Low setzt, erzeugt der Buzzer einen Ton, entsprechend der Frequenz. Es gibt verschiedene Möglichkeiten, den Buzzer anzusteuern. Im Folgenden werden unterschiedliche Methoden erläutert.

```
1      Buzzer buzzer;           //Initialisierung
2      ...
3      buzzer.playFrequency(frequenz, dauer, lautstaerke);
```

Listing 6: Buzzer-Ansteuerung mit PlayFrequency

Die übergebenen Parameter sind für die Funktion `playFrequency()` die Frequenz, Dauer und Lautstärke. Die Frequenz wird in Hz angegeben, oder 0,1 Hz, falls das `DIV_BY_10` Bit gesetzt wird. Das Argument für die Frequenz muss zwischen 40 Hz und 10 kHz liegen. Wenn man beispielsweise aber eine Frequenz von 44,5 Hz spielen will, kann man folgende Syntax verwenden (`DIV_BY_10 | 445`). Die Dauer wird in Millisekunden angegeben. Dabei ist zu berücksichtigen, dass die Zeit nicht größer als 0xFFFF (65 535 ms) sein darf. Die Lautstärke hat einen Wertebereich von 0 bis 15, wobei 15 die maximale und 0 die minimale Lautstärke darstellt. Die Einstellung 15 versorgt den Buzzer mit einem Tastgrad von 50 %. Der Tastgrad wird mit jeder Stufe halbiert, Stufe 14 entspricht also 25 % Tastgrad, Stufe 13 entspricht 12,5 % usw. Green, 2024 In Listing 7 wird die Ansteuerung des Buzzers gezeigt:

```
1      Buzzer buzzer;           //Initialisierung
2
3      // spiele eine 5 kHz note fuer 200 ms in einer leiseren
4      // Lautstaerke
5      buzzer.playFrequency(5000, 200, 12);
6
7      // spiele eine 44,5 Hz Note fuer 500 ms mit voller Lautstaerke
8      buzzer.playFrequency(DIV_BY_10 | 445, 500, 15);
```

Listing 7: Buzzer Ansteuerung, Green, 2024

Die Note wird so lange gespielt, wie man als Parameter angegeben hat. Wird erneut eine Buzzerfunktion aufgerufen während eine Note gespielt wird, überschreibt der neue Funktionsaufruf den vorherigen und steuert den Buzzer.

Es können aber auch spezifische Noten gespielt werden, welche man über die `playNote(Note, Dauer, Lautstärke)` Funktion aufrufen kann. Bei dieser Funktion kann man eine spezifische Note übergeben, welche dann gespielt wird. Die Noten sind als Characters spezifiziert: C; D; E; F; G; A; B. Diese werden als Grundeinstellung als Viertelnoten gespielt, das entspricht einer Zeit von 500 ms, also 120 beats/min (bpm).

Wenn man Noten aneinanderreihen möchte, also eine Melodie spielen möchte, dann sollte man eine Sequenz abspielen mit der `play(*Noten)`-Funktion. Diese Funktion spielt eine Notenfolge im Hintergrund, während das Programm weiterläuft. Beim Abspielen einer Sequenz ist es sinnvoll, zu warten, bis sie abgeschlossen ist, bevor die Buzzerfunktion erneut aufgerufen wird. Andernfalls wird der vorherige Aufruf überschrieben, und der neue übernimmt die Steuerung des Buzzers. Dafür gibt es eine Funktion `isPlaying()` die überprüft, ob aktuell eine Sequenz abgespielt wird oder nicht. Sie gibt eine 0 (false) zurück,

wenn die Sequenz, Note, Frequenz abgeschlossen ist, und eine 1 (true), wenn die Sequenz noch läuft. Nachfolgend werden in Tabelle 5 die einstellbaren Parameter einer Sequenz dargestellt sowie ein Syntaxbeispiel in Listing 8 für den Aufbau einer Sequenz:

Tabelle 5: Buzzer-Einstellmöglichkeiten für die Funktion play()

Befehl	Beschreibung
A-G	Bestimmt eine Note, die gespielt wird.
R	Pause (Kein Ton für die Dauer der Note).
+ oder # nach einer Note	Erhöht die vorherige Note um einen Halbton.
- nach einer Note	Erniedrigt die vorherige Note um einen Halbton.
1-2000 nach einer Note	Bestimmt die Dauer der vorherigen Note (z.B. C16 für eine Sechzehntelnote).
. nach einer Note	Verlängert die vorherige Note um 50%. Mehrere Punkte erhöhen die Dauer weiter (z.B. A.. ist 1,75-mal so lang wie A).
> vor einer Note	Spielt die nächste Note eine Oktave höher.
< vor einer Note	Spielt die nächste Note eine Oktave tiefer.
O gefolgt von einer Zahl	Setzt die Oktave (Standard: O4).
T gefolgt von einer Zahl	Setzt das Tempo in BPM (Standard: T120).
L gefolgt von einer Zahl	Setzt die Standard-Notendauer (L4 für Viertelnoten, L8 für Achtelnoten usw., Standard: L4).
V gefolgt von einer Zahl	Setzt die Lautstärke (0-15, Standard: V15).
MS	Setzt Staccato (jede Note wird nur zur Hälfte gespielt, danach Stille).
ML	Setzt Legato (jede Note wird in voller Länge gespielt, Standard).
!	Setzt Oktave, Tempo, Dauer, Lautstärke und Staccato-Einstellungen auf Standardwerte zurück.

Green, 2024

```

1  Buzzer buzzer;           //Initialisierung
2
3  //Sequenz
4  buzzer.play("!L16 V8 cdefgab>cbagfedc");
5
6  //Ueberpruefung ob die Sequenz fertig gespielt wurde
7  while (buzzer.isPlaying());
8
9  //Sequenz
10 buzzer.play("!T240 L8 agafaea dac+adaea fa<aa<bac#a dac#adaea f4");

```

Listing 8: Buzzer Sequenz mit Überprüfung,

Green, 2024

Wenn man möchte, dass der Buzzer sofort aufhört zu spielen, kann man die `stopPlaying()` Funktion aufrufen. Wie der Name schon sagt, hört der Buzzer bei diesem Funktionsaufruf sofort auf zu spielen, egal, welche Funktion den Buzzer gerade steuert.

Der Buzzer kann als akustisches Feedback für ein Programm genutzt werden. Er ermöglicht es, durch Töne den aktuellen Status des Roboters hörbar zu machen. So könnte beispielsweise ein Ton beim Start des Roboters erklingen, ein weiterer beim Erkennen eines Abzweigs oder eines Hindernisses. Ebenso kann ein Signal ertönen, wenn der Roboter seine Aufgabe, nach seinem Verständnis, erfolgreich abgeschlossen hat. Auch beim Drücken eines Schalters kann ein Ton als Rückmeldung abgespielt werden. Die genaue Nutzung des Buzzers bleibt dabei dem Anwender überlassen.

3.3 Auslesen und Verarbeiten der Sensoren

Encoder auslesen

Um an die gemessenen Counts zu kommen, kann man sich die Werte vom Microcomputer ausgeben lassen. Im folgenden Beispiel sehen sie einen einfachen Code unter Verwendung der Bibliothek um sich die Counts in Variablen speichern zu lassen.

```
1  Encoders encoders; // globale deklaration
2  ...
3  Int countLeft=encoders.getCountsLeft(); // Speichert den
   Encoderwert in eine Variable
4  Int countRight=encoders.getCountsRight();
5  ...
6  Int countLeft=encoders.getCountsAndResetLeft(); // Speichert
   Wert in eine Variable
7  Int countRight=encoders.getCountsAndResetRight(); // Und loescht
   dazugehoerigen den Speicher
```

Listing 9: Wertausgabe der Counts

Für eine akkurate Steuerung sind präzise Ansteuerungskomponenten absolut wichtig, da die Qualität der Ansteuerung direkt die Genauigkeit der Steuerung beeinflusst. Der Roboter muss in der Lage sein, sich in dem Labyrinth auf der Stelle zu drehen, Hindernisse erkennen, und ein gleichmäßig geregelten Antrieb haben, um gut durch das Labyrinth navigieren zu können. Green, 2024

Ansteuerung der Stoßsensoren

In diesem Kapitel wird auf die Verwendung der Stoßsensoren eingegangen. Es gibt verschiedene Funktionen wie man auf die Sensoren zugreifen kann.

```
1  //Initialisierung
2  BumpSensors bumpSensors;
3
4  bumpSensors.calibrate();
5  bumpSensors.read();
6  bumpSensors.leftChanged();
7  bumpSensors.rightChanged();
8  bumpSensors.leftIsPressed();
9  bumpSensors.rightIsPressed();
```

Listing 10: Funktionen für den Stoßsensor,

Green, 2024 Die Funktion `calibrate()` sollte zu Beginn des Programms aufgerufen werden, um sicherzustellen, dass die Stoßsensoren korrekt kalibriert sind. Ohne Kalibrierung könnten die Sensoren falsche oder ungenaue Werte liefern. Die Funktion führt die Kalibrierung durch, indem sie eine bestimmte Anzahl von Messwerten erfasst (standardmäßig 50) und daraus einen Mittelwert (Baseline) berechnet.

Die Variable `marginPercentage` gibt die Toleranz an und ist in der Bibliothek mit einem Standardwert von 50 festgelegt. Dieser Wert muss in der Regel nicht geändert werden, da die Voreinstellung bereits gut funktioniert. Die Kalibrierung erfolgt, während die Sensoren nicht gedrückt sind.

$$\text{threshold} = \text{baseline} \times \frac{100 + \text{marginPercentage}}{100} \quad (1)$$

Green, 2024

Mithilfe dieser Formel wird der Schwellenwert (`threshold`) berechnet, ab dem der Sensor als “gedrückt,” oder “nicht gedrückt,” erkannt wird.

Die Funktion `read()` gibt beispielsweise einen Rückgabewert von 1 zurück, was binär 0b01 entspricht. In diesem Fall steht das höchstwertige Bit (MSB) rechts. Das bedeutet, dass der rechte Stoßsensor aktiviert ist, da Bit 0 (`BumpRight`) gesetzt ist. Der linke Stoßsensor ist hingegen nicht aktiviert, da Bit 1 (`BumpLeft`) nicht gesetzt ist.

Anstatt den Rückgabewert von `read()` manuell auszuwerten, können anschließend die Funktionen `leftChanged()`, `rightChanged()`, `leftIsPressed()` und `rightIsPressed()` genutzt werden, um den Status der Stoßsensoren direkt abzufragen. Green, 2024

Die Stoßsensoren können als zusätzliche Sicherheitseinheit eingesetzt werden. In dieser Größenordnung sind Schäden zwar eher selten, dennoch ist es gute Praxis, den Roboter so gut wie möglich vor Beschädigungen zu schützen. Sollte der Time-of-Flight-Sensor eine Wand oder ein anderes Hindernis nicht erkennen, können die Stoßsensoren als Sicherheitsmaßnahme dienen. Sie stoppen den Roboter oder schalten die Motoren ab, sobald ein Stoßsensor aktiviert wird.

Programmierung und Ansteuerung der IMU (Magnet- und Beschleunigungssensor)

Die Sensorik der IMU umfasst sowohl den Magnet- als auch den Beschleunigungssensor und wird innerhalb der verwendeten Bibliothek zusammengefasst. Um diese Sensorik über die I²C-Schnittstelle anzusteuern oder auszulesen, muss zunächst die I²C-Kommunikation initialisiert werden. Dies geschieht durch den Aufruf von `Wire.begin(Baudrate)`. In Listing 11 sind Funktionen für die IMU aufgelistet:

```
1  IMU imu;
2  imu.init();
3  imu.getLastError();
4  imu.enableDefault();
5  imu.configureForTurnSensing();
6  imu.configureForFaceUphill();
7  imu.configureForCompassHeading();
8  imu.writeReg(uint8_t addr, uint8_t reg, uint8_t value);
9  imu.readReg(uint8_t addr, uint8_t reg);
10 imu.readAcc();
11 imu.readGyro();
12 imu.readMag();
13 imu.read();
14 imu.accDataReady();
15 imu.gyroDataReady();
16 imu.magDataReady();
```

Listing 11: Funktionen für den IMU-Sensor, Green, [2024](#)

Die Funktionen im Überblick:

- **init()**: Initialisiert die IMU und bereitet sie für den Betrieb vor.
- **getLastError()**: Liefert eine 0 zurück, wenn die letzte I²C-Kommunikation erfolgreich war, andernfalls einen Fehlercode.
- **enableDefault()**: Aktiviert die IMU-Sensoren mit den Standardkonfigurationen.
- **configureForTurnSensing()**: Optimiert die Einstellungen für die Erkennung von Drehbewegungen.
- **configureForFaceUphill()**: Passt die Sensorkonfiguration für das Beispielprogramm „FaceUphill“ an.
- **configureForCompassHeading()**: Konfiguriert den Sensor für die Erkennung magnetischer Richtungen.
- **writeReg(addr, reg, value)**: Schreibt einen Wert in ein 8-Bit-Register des Sensors.
- **readReg(addr, reg)**: Liest den Wert eines bestimmten Registers aus.
- **readAcc()**: Erfasst die aktuellen Beschleunigungswerte in Form eines Vektors (x, y, z).
- **readGyro()**: Liest die Winkelgeschwindigkeit aus dem Gyroskop aus.
- **readMag()**: Gibt die aktuellen Magnetfeldmesswerte zurück.

- **read()**: Liest die Sensordaten aller drei Sensoren (Beschleunigung, Gyroskop und Magnetometer) aus.
- **accDataReady()**: Prüft, ob neue Beschleunigungsdaten verfügbar sind.
- **gyroDataReady()**: Überprüft, ob aktuelle Gyroskopdaten bereitstehen.
- **magDataReady()**: Testet, ob neue Magnetometerwerte verfügbar sind.

Green, [2024](#)

Ansteuerung der Liniensensoren

Bevor die Liniensensoren des *Pololu 3pi+ 2040* für die Linienverfolgung verwendet werden können, ist eine Kalibrierung notwendig. Ziel dieser Kalibrierung ist es, für jeden der fünf Sensoren den jeweils minimalen und maximalen Messwert zu bestimmen. Diese Werte dienen später als Referenz, um Rohdaten der Sensoren in normierte, kalibrierte Werte im Bereich von 0 bis 1000 zu überführen. Der Wertebereich ist dabei fest definiert: Ein Sensorwert von 0 entspricht dem niedrigsten bei der Kalibrierung gemessenen Wert (z. B. über einem hellen Untergrund), und ein Wert von 1000 entspricht dem höchsten gemessenen Wert (z. B. über einer schwarzen Linie). Diese Skalierung wird automatisch durch die Methode **readCalibrated()** vorgenommen.

Die Kalibrierung erfolgt über die Methode **calibrate()** aus der Klasse **LineSensors**. Die Methode liest die Sensoren 10 mal, wobei standardmäßig die IR-LEDs eingeschaltet sind. Der Roboter sollte sich während der Kalibrierung auf der Linie befinden und bewegt werden, da sonst keine unterschiedlichen Reflexionen, außer Anomalien der Oberfläche, festgestellt werden können. Durch die Bewegung erfasst jeder Sensor mindestens einmal die Linie und den Hintergrund. Die Werte werden dann für jeden Sensor einzeln automatisch gespeichert. Dadurch, dass die Minima und Maxima der Sensoren einzeln abgespeichert werden, spielen durch die Normierung die Toleranzen keine Rolle mehr. Green, [2024](#)

Wichtig ist, dass der Kalibriermodus **LineSensorsReadMode::Manual** hierbei **nicht** verwendet werden darf. In diesem Modus werden die Infrarot-Emitter nicht automatisch gesteuert, was zu ungültigen Kalibrierwerten führen kann. Die empfohlene Einstellung ist daher **LineSensorsReadMode::On**, sodass die Sensoren ihre Messungen mit eingeschalteten Emittlern durchführen. Diese Einstellung ist aber standardmäßig aktiv, sofern man keine Änderungen vorgenommen hat, muss man dies nicht beachten. Mit der Funktion **resetCalibration()** werden alle Kalibrierwerte zurückgesetzt.

Nach abgeschlossener Kalibrierung können die Sensordaten durch Aufruf von **readCalibrated()** aufgerufen werden. Die Funktion gibt die kalibrierten Sensordaten in einem Wertebereich von 0 bis 1000 zurück, wie zuvor schon beschrieben. Green, [2024](#)

Die Methode **readLineBlack()** nutzt die kalibrierten Werte, um die Position einer schwarzen Linie unter den Sensoren zu bestimmen. Die Sensoren sind wie ein Array unten an dem Roboter befestigt und können von links nach rechts von 0 bis 4 durchgezählt werden.

Es wird eine gewichtete Mittelwertberechnung durchgeführt, bei der jeder Sensor je nach Messwert stärker oder schwächer zur Positionsbestimmung beiträgt. Das Ergebnis ist ein ganzzahliger Wert zwischen 0 (Linie vollständig unter dem linken Sensor) und 4000 (Linie vollständig unter dem rechten Sensor). Sollte sich die Linie zwischen zwei Sensoren befinden, liefert die Methode Zwischenwerte wie 1500 oder 2750. Dabei ändert sich der Wert stetig, wenn die Linie sich verschiebt. Die geschätzte Position der Linie wird mit folgender Formel berechnet:

$$\text{Position} = \frac{(0 \cdot v_0) + (1000 \cdot v_1) + (2000 \cdot v_2) + (3000 \cdot v_3) + (4000 \cdot v_4)}{v_0 + v_1 + v_2 + v_3 + v_4}$$

Green, 2024

- v_0 bis v_4 : Messwerte der Sensoren 0 bis 4
 - v_0 : Sensor ganz links
 - v_4 : Sensor ganz rechts
- Die Multiplikatoren (0, 1000, 2000, 3000, 4000) geben die Position der Sensoren an.
- Der Rückgabewert **Position** ist ein gewichteter Mittelwert, der angibt, wo sich die Linie relativ zur Sensorleiste befindet:
 - Wert nahe 0 → Linie unter Sensor 0 (ganz links)
 - Wert nahe 4000 → Linie unter Sensor 4 (ganz rechts)
 - Zwischenwerte → Linie liegt zwischen zwei Sensoren

Die von der Bibliothek bereitgestellte Methode zur Linienverfolgung eignet sich aus mehreren Gründen hervorragend für die Implementierung einer PID-Regelung. Die wesentlichen Vorteile lassen sich in den folgenden Eigenschaften zusammenfassen.

Ein Vorteil ist die Generierung einer kontinuierlichen und fein aufgelösten Positionsinformation. Anstatt lediglich den diskreten Index eines aktiven Sensors zurückzugeben, wird durch eine gewichtete Mittelwertbildung eine genaue Position der Linie, auch zwischen zwei Sensoren, ermittelt. Durch die Multiplikation der Sensorindizes mit dem Faktor 1000 wird die Auflösung weiter erhöht, was zu einer breiten Werte-Spanne (z. B. 0 bis 4000 bei fünf Sensoren) führt. Diese hohe Auflösung ist insbesondere für den Differentialanteil (D-Anteil) des PID-Reglers vorteilhaft, da selbst minimale Abweichungen von der Solltrajektorie präzise und zeitnah erfasst werden können. Das Ergebnis ist ein stetiges und geglättetes Positionssignal.

Die Positionsschätzung der Methode ist monoton. Dies bedeutet, dass eine gleichmäßige Verschiebung der Linie unter der Sensorleiste zu einem proportional ansteigenden oder fallenden Rückgabewert führt. Dieses Verhalten verhindert abrupte Sprünge im Messsignal und ermöglicht dem Regler eine zuverlässige Berechnung des Fehlersignals sowie seiner zeitlichen Änderung (Ableitung), was die Stabilität des Regelkreises verbessert.

Für den Fall, dass der Roboter die Linie vollständig verlässt, speichert die Methode die Position des zuletzt erfassten Liniensegments. Verliert der Roboter beispielsweise die Führung, weil er zu weit nach links gefahren ist, gibt die Funktion weiterhin den Maximalwert (z. B. 4000) zurück. Dieser Wert informiert den Regler darüber, auf welcher Seite sich die Linie zuletzt befand. Dadurch kann die Regelung aktiv versuchen, den Roboter zur Linie zurückzusteuern, anstatt orientierungslos zu stoppen. Diese Eigenschaft erhöht die Robustheit des Gesamtsystems erheblich Green, 2024.

Für die Verfolgung einer weißen Linie auf dunklem Untergrund stellt die Bibliothek die analoge Methode `readLineWhite()` zur Verfügung, welche mit invertierten Sensorwerten arbeitet. Das grundlegende Prinzip der Positionsbestimmung bleibt dabei identisch.

Obwohl für die Regelung primär die kalibrierten Werte (`readCalibrated()`) oder die direkten Linienpositionswerte genutzt werden sollten, bieten die Funktionen `read()` und `rawSensorValues` Zugriff auf die Rohdaten der Sensoren. Dies kann insbesondere bei der Fehlersuche und Kalibrierung hilfreich sein Green, 2024.

Die Implementierung der Linienverfolgung dient im Rahmen dieser Arbeit als alternative Navigationsmethode, die unabhängig von Distanzsensoren wie dem Time-of-Flight-Sensor agiert. Als vollständig auf der Hardware integrierte Onboard-Lösung erfordert sie keine externen Komponenten. Grundvoraussetzung für eine zuverlässige Funktion ist jedoch eine klar definierte Fahrspur, die einen deutlichen Kontrast zum Untergrund aufweist.

Auslesen des ToF-Sensors OPT3101

Für die Verwendung des Sensors wird die [GitHub-Bibliothek Opt3101](#) von Pololu empfohlen. Dieser Link steht auch im Anhang H zur Verfügung. Grayson et al., 2020

```
1  void setAddress(uint8_t address) { this->address = address; }
2  uint8_t getAddress() { return address; }
3  uint8_t getLastError() { return lastError; };
4  void resetAndWait();
5  void writeReg(uint8_t reg, uint32_t value);
6  uint32_t readReg(uint8_t reg);
7
8  void init();
9  void configureDefault();
10 void setChannel(uint8_t channel);
11 void nextChannel();
12 void setBrightness(OPT3101Brightness);
13 void setMonoshotMode();
14 void setContinuousMode();
15 void setFrameTiming(uint16_t subFrameCount);
16 void enableTimingGenerator();
17 void disableTimingGenerator();
18 void enableDataReadyOutput(uint8_t gpPin);
19 void startSample();
20 bool isSampleDone();
21 void readOutputRegs();
22 void sample();
23
24 uint8_t channelUsed = 0;
25 OPT3101Brightness brightnessUsed;
26 uint16_t ambient;
27 uint16_t temperature;
28 int32_t i, q;
29 uint16_t amplitude;
30 int16_t phase;
31 int16_t distanceMillimeters;
```

Listing 12: Funktionen und Variablen der Klasse für den ToF-Sensor, Green, 2024

- **void setAddress:** Mit dieser Funktion kann man eine eigene Adresse festlegen. Dies ist aber nicht nötig, solange es keine anderen Komplikationen mit andern I²C - Adressen gibt. Standardmäßig ist 0x58 eingestellt.
- **uint8_t getAddress():** Gibt den Wert der aktuellen Adresse zurück.
- **uint8_t getLastError():** Gibt den letzten Fehler zurück.
- **void resetAndWait():** Setzt den Sensor in einen definierten Zustand zurück.
- **void writeReg:** Mit dieser Methode kann man Werte in ein Register schreiben.
- **uint32_t readReg:** Liest das übergebene Register aus.
- **void init():** Initialisiert den Sensor und konfiguriert ihn mit den Standardkonfigurationen. Die Methode sollte vor jedem Start oder nach jedem Reset aufgerufen werden um den Sensor startklar zu machen.
- **configureDefault():** konfiguriert den Sensor mit einer Reihe von Standardwerten. Dies wird auch durch init aufgerufen und muss daher nicht extra aufgerufen werden.
- **void setChannel(uint8_t channel);:** gibt an, welcher Channel verwendet werden soll. 0,1,2 -> Auswahl eines festen Kanals, jder andere Wert -> Automatischer Kanalwechsel (Autoswitch)
- **void nextChannel():** Die Methode schaltet zyklisch auf den nächsten verfügbaren Kanal des OPT3101-Sensors um. Der Sensor hat drei Kanäle (0, 1, 2), zwischen denen man manuell wechseln kann.
- **void setBrightness:** legt den Helligkeitsmodus der LED's fest, diese Konfigurationen gibt es: Low; high; adaptive. Adaptive passt sich den Verhältnissen automatisch an.
- **void setMonoshotMode():** Konfiguriert den OPT3101 für den Monoshot-Modus, in dem nur ein einzelnes Frame pro Auslösung aufgenommen wird, anstatt kontinuierlich zu messen.
- **void setContinuousMode():** Stellt den Sensor auf kontinuierliche Messung um, sodass fortlaufend Frames aufgenommen werden, ohne manuelles Auslösen.
- **void setFrameTiming(uint16_t subFrameCount):** Setzt die Anzahl und Dauer von Subframes, wodurch die Gesamtdauer eines Mess-Frames bestimmt wird.
- **void enableTimingGenerator():** Aktiviert den internen Zeitgeber des Sensors, indem sie das Register 0x80 setzt.
- **void disableTimingGenerator():** Deaktiviert den internen Zeitgeber des Sensors.
- **void enableDataReadyOutput(uint8_t gpPin):** Aktiviert ein Data-Ready-Signal auf GPIO-Pin 1 oder 2, damit ein Mikrocontroller erkennen kann, wann neue Daten zum Auslesen bereitstehen.
- **void startSample();:** Führt eine Einzelmessung durch.

- **bool isSampleDone():** Überprüft, ob eine gestartete Einzelmessung (Sample) abgeschlossen ist. Sie gibt ein True zurück wenn die Messung fertig ist und False wenn die Messung noch läuft.
- **void readOutputRegs():** liest die Messergebnisse und Statusinformationen vom OPT3101-Sensor aus den entsprechenden Registern (0x08, 0x09, 0x0a, 0x3b, 0x3c) aus.
- **void sample():** führt eine vollständige Einzelmessung des Sensors durch – von Start bis Datenauswertung.
 - Kanal (channelUsed) und Helligkeitsmodus (brightnessUsed)
 - IQ-Daten: i, q – Rohwerte für Signalverarbeitung
 - Amplitude (amplitude) und Phase (phase) des reflektierten Lichts
 - Entfernung in Millimeter (distanceMillimeters), berechnet aus der Phase
 - Umgebungslichtwert (ambient)
 - Temperaturwert (temperature) des Sensors

Die folgenden Variablen sind Teil der Klasse **OPT3101** und dienen zur Zwischenspeicherung der ausgelesenen Daten aus den Registern des Sensors. Es handelt sich dabei nicht um direkte Registeradressen, sondern um Programmvariablen, die abgerufen werden können:

- **uint8_t channelUsed**
Gibt an, welcher der drei Kanäle (0–2) zuletzt verwendet wurde.
- **OPT3101Brightness brightnessUsed**
Speichert den genutzten Helligkeits- bzw. Belichtungsmodus (z. B. adaptive HDR oder feste Helligkeit).
- **uint16_t ambient**
Enthält den gemessenen Umgebungslichtwert (aus Register 0x0A, Bits 2–11).
- **uint16_t temperature**
Interner Temperaturwert des Sensors (aus Register 0x0A, Bits 12–23).
- **int32_t i, q**
Rohwerte der In-Phase- und Quadraturkomponenten des reflektierten Signals (aus den Registern 0x3B und 0x3C). Diese Werte werden ggf. ins Zweierkomplement umgerechnet.
- **uint16_t amplitude**
Signalstärke des reflektierten Lichts (aus Register 0x09, Bits 0–15).
- **int16_t phase**
Phasenverschiebung des Empfangssignals (aus Register 0x08, Bits 0–15).
- **int16_t distanceMillimeters**
Daraus berechnete Entfernung zum Objekt in Millimetern, basierend auf der Phase.

3.4 Ansteuerung des OLED-Displays

Das auf der Hauptplatine verbaute OLED-Display dient als primäres Ausgabemedium für Statusinformationen, Messwerte oder Menüführungen direkt am Roboter. Die Ansteuerung erfolgt über die Pololu-Bibliothek, welche die komplexe SPI-Kommunikation mit dem SH1106-Controller abstrahiert und benutzerfreundliche Funktionen zur Verfügung stellt.

Die grundlegende Initialisierung und Ansteuerung umfasst typischerweise die folgenden Schritte:

1. **Initialisierung:** Mit dem Aufruf von `display.init()` wird das Display zurückgesetzt, der interne Speicher gelöscht und Standardeinstellungen wie Kontrast und Orientierung geladen.
2. **Display-Puffer löschen:** Der Befehl `display.clear()` leert den Textpuffer und setzt den Cursor an die Startposition (oben links).
3. **Cursor positionieren:** Mit `display.gotoXY(spalte, zeile)` kann der Cursor an eine beliebige Stelle im Text-Grid des Displays bewegt werden.
4. **Text ausgeben:** Über `display.print("Text")` oder `display.print(variable)` werden die gewünschten Informationen in den Puffer geschrieben.
5. **Anzeige aktualisieren:** Die Pololu-Bibliothek arbeitet mit einem Auto-Display-Modus, bei dem Änderungen oft direkt sichtbar werden. Für flackerfreie Aktualisierungen kann dieser Modus mit `display.noAutoDisplay()` deaktiviert werden. Nach dem Schreiben aller Änderungen in den Puffer erzwingt der Befehl `display.display()` eine einmalige, vollständige Aktualisierung des Bildschirms.

Das folgende Code-Beispiel demonstriert die grundlegende Verwendung zur Anzeige eines einfachen Textes:

```
1  #include <Pololu3piPlus2040.h>
2  // Der Namespace der Bibliothek wird eingebunden.
3  // Das Objekt "display" ist dadurch global verfuegbar.
4  using namespace Pololu3piPlus2040;
5  void setup() {
6      // Das Display wird zu Beginn einmal komplett geloescht.
7      display.clear();
8      // Statischer Text wird in die erste Zeile geschrieben.
9      display.gotoXY(0, 0);
10     display.print("Status:");
11 }
12 void loop() {
13     // Eine statische Variable, die nur einmal initialisiert wird
14     // und ihren Wert zwischen den loop-Aufrufen behaelt.
15     static int zaehler = 0;
16     // Setze den Cursor in die zweite Zeile, hinter einen Text.
17     display.gotoXY(0, 1);
18     display.print("Zaehler: ");
19     // Gib den aktuellen Wert der Variable aus.
20     display.print(zaehler);
21     display.print("  ");
22     // Erhoehe den Zaehler fuer den naechsten Durchlauf.
23     zaehler++;
24     delay(1000);}
```

Listing 13: Dynamische Zähler-Ausgabe auf dem OLED-Display

Neben der reinen Textdarstellung ermöglicht die Bibliothek auch die Konfiguration verschiedener Layouts (z.B. 8×2 oder 21×8 Zeichen) sowie die Kombination von Text mit einer externen Grafik-Puffer-Variable, um eigene Grafiken oder Symbole darzustellen. Weiter Informationen zu weiteren Funktionen sind auf der GitHub-Seite nachzulesen. Grayson et al., [2021](#)

3.5 Interaktion und Debugging

Programmierung der Taster

Auf der Platine des Roboters befinden sich fest verbaute Taster, von denen drei programmierbar sind bzw. der Status abgefragt werden kann. Der grün umrahmte Taster in Abbildung 12 dient zum Ein- und Ausschalten des Roboters. Sobald eine Stromversorgung über USB-C hergestellt wird, ist der Roboter dauerhaft eingeschaltet. Mit dem grün umrahmten Taster lassen sich die Motoren ein- oder ausschalten – allerdings nur, wenn zusätzlich Batterien im Batteriefach eingelegt sind und den Roboter mit Strom versorgen können. Der rechts in Abbildung 12 befindliche Taster ist der Reset-Taster. Er unterbricht kurz die Stromversorgung und setzt dadurch das Programm zurück bzw. erzeugt einen Neustart.

Die mittig über dem USB-C-Anschluss rot umrahmten Taster A, B und C (siehe Abbildung 12) können programmiert und über die Funktion `isPressed()` abgefragt werden. Im folgenden Listing 14 sehen Sie, wie diese Taster in den Code eingebunden werden können:

```
1 //Initialisieren
2 ButtonA buttonA;
3 ButtonB buttonB;
4 ButtonC buttonC;
5
6 //Tasterstatus abfragen
7 if(buttonA.isPressed()){};
8 if(buttonB.isPressed()){};
9 if(buttonC.isPressed()){};
```

Listing 14: Taster einbinden

Wird beim Einschalten des Roboters die Taste B gedrückt gehalten, wechselt das Gerät in den Bootloader-Modus. Dieser Modus ermöglicht es, neuen Code auf den Mikrocontroller zu übertragen, auch dann, wenn das Gerät zuvor nicht mehr programmierbar war. Pololu, [2024c](#), S. 6.2

Der Bootloader-Modus ist insbesondere in zwei Situationen hilfreich: Erstens beim Wechsel vom werkseitig vorinstallierten Python-Code auf eine C++ basierte Entwicklung; zweitens, wenn der Mikrocontroller durch einen fehlerhaften Code blockiert ist und sich auf normalem Weg nicht mehr programmieren lässt. In beiden Fällen kann der Bootloader manuell aktiviert und anschließend ein neuer Sketch erfolgreich aufgespielt werden.

Die Taster können verwendet werden, um durch ein Menü zu navigieren oder ein bestimmtes Programm zu starten. Zur Menüführung kann das Display genutzt werden. Es bleibt dem Anwender überlassen, ob und wie die Taster in das Programm eingebunden werden.

Nutzung der seriellen Schnittstelle

Die serielle Schnittstelle des Mikrocontrollers stellt ein äußerst nützliches Werkzeug zur Kommunikation zwischen Mikrocontroller und PC dar. Sie eignet sich insbesondere für das Debugging sowie für die Echtzeit-Ausgabe von Variablenwerten. Dadurch wird die Entwicklung und Fehlersuche im Code deutlich erleichtert und effizienter gestaltet.

Um die serielle Schnittstelle nutzen zu können, muss der Mikrocontroller über ein USB-Kabel mit dem Computer verbunden sein. Die serielle Kommunikation ermöglicht es beispielsweise, Sensordaten oder Encoderwerte auszulesen, insbesondere dann, wenn keine direkte Anzeige über ein Display vorgesehen ist. Stattdessen können die Daten über den seriellen Monitor der Arduino-IDE eingesehen werden. Der serielle Monitor fungiert als Terminalfenster: Er stellt empfangene Daten in Echtzeit dar und erlaubt darüber hinaus auch das Senden von Befehlen an den Mikrocontroller.

Um die serielle Kommunikation zu starten, muss im Arduino-Code die serielle Schnittstelle initialisiert werden. Dies geschieht mit `Serial.begin(9600);`, wobei die Baudrate – in diesem Fall 9600 Baud – vom Anwender definiert wird. Anschließend können mit `Serial.println()` Nachrichten an den seriellen Monitor gesendet werden.

Das folgende Beispiel zeigt, wie eine einfache serielle Kommunikation mit dem Arduino eingerichtet werden kann:

```
1  void setup() {
2      // Starten der seriellen Kommunikation bei 9600 Baud
3      Serial.begin(9600);
4      int Variable = 1;
5      delay(1000);
6
7      // Initiale Nachricht, die nur einmal gesendet wird
8      Serial.println("Serielle Verbindung gestartet!");
9  }
10
11 void loop() {
12     // Nachricht an den seriellen Monitor senden
13     Serial.println("Test des seriellen Monitors erfolgreich");
14     Serial.print(Variable);
15     // 1 Sekunde warten
16     delay(1000);
17 }
```

Listing 15: Serielle Kommunikation mit dem Arduino

In diesem Code wird beim Start eine einmalige Meldung ausgegeben, die signalisiert, dass die serielle Verbindung hergestellt wurde. Danach wird in der Endlosschleife (`loop()`) jede Sekunde eine Nachricht gesendet. Wird dieser Sketch auf den RP2040 hochgeladen und der serielle Monitor in der Arduino-IDE geöffnet, kann die Ausgabe in Echtzeit verfolgt werden. Wenn man nun eine Variable hat, die man sich in Echtzeit anzeigen lassen will, kann man sich diese nun über den seriellen Monitor ausgeben lassen. Dies macht auch Sinn wenn man Sensoren kalibriert. So kann man sich direkt unter den gegebenen Bedingungen Minima und Maxima anzeigen lassen, um einen Wertebereich festzulegen.

Nutzt man Visual-Studio-Code so muss man vorher in seinem Projekt in der Datei "platformio.ini" diese Zeile hinzufügen: `monitor_speed = 9600` //Einstellbare Baudrate.

4 Anwendungsfall: Das Labyrinth

4.1 Regelwerk des offiziellen Micromouse-Contest von IEEE

Teilnahmeberechtigung

1. IEEE Mitgliedschaft:

Als Teilnehmer muss man als IEEE-Student an der Universität eingeschrieben sein bei man beim Student Activities Committee (SAC) registriert ist. IEEE, [2020](#), S.1 2.1

2. Zusammensetzung des Teams:

Die Teams bestehen aus einem bis fünf Teilnehmern. Ein Team mit vier oder fünf Teilnehmern darf maximal zwei Doktoranden enthalten. Ein Team mit zwei oder drei Teilnehmern darf maximal einen Doktoranden enthalten. Teams, die ausschließlich aus promovierten Studenten bestehen, sind nicht zugelassen. Im Falle eines Teams mit mehr als einem Teilnehmer muss nachgewiesen werden, dass jeder Teilnehmer einen wesentlichen Beitrag geleistet hat. IEEE, [2020](#), S.1 2.2

3. Anzahl der Teams:

Es gibt keine Teambegrenzung, jede Universität kann so viele Teams anmelden, wie sie möchten. IEEE, [2020](#), S.1 2.3

Regeln für den Roboter

1. Bau des Roboters:

Der Micromouseroboter, mit dem an dem Wettbewerb angetreten wird, muss von Null auf vom anzutretenden Team selber geplant und gebaut werden, also ohne Vorlagen, Frameworks oder bereits bestehende Komponenten zurückgreifen. IEEE, [2020](#), S.2 Kap. 3.1

Unter dieser Regel ist jedoch nicht definiert, in welchem Detailgrad die Hardware selber gebaut werden muss. Beispielsweise muss das Rad selber produziert werden über Fräsen oder 3D-Drucker?

Daher ist anzunehmen, dass die Bauteile aus der letztmöglichen Ebene, also dem Grundzustand gekauft werden dürfen. Sprich, es darf keine Zusammensetzung aus Bauteilen gekauft werden. Beispielsweise muss die Platine selber designt werden, die Chips und elektronischen Komponenten werden gekauft und vom Team verlötet. Es darf jedoch keine bereits existierende Platine gekauft werden, die nur noch programmiert werden muss.

2. Selbstständigkeit:

Die Micromouse muss eigenständig funktionieren und darf nicht durch externe Steuerung oder Fernbedienungen gesteuert werden, sprich, er muss autonom fahren. Der Roboter darf keine Energiequellen verwenden, die auf einem Verbrennungsprozess (z.B. Benzin- oder Dieselmotoren) basieren. IEEE, [2020](#), S.2 3.2

3. Keine gelösten Teile:

Während der Roboter durch das Labyrinth fährt, darf er kein Teil verlieren oder abtrennen. Der Roboter muss eine Einheit bleiben. Um die Aufgabe erfolgreich zu erfüllen, muss der gesamte Roboter in einer vollständigen Form, also mit allen Komponenten die Mitte des Labyrinths erreichen. IEEE, 2020, S.2 3.3

4. Bewegungsmethoden:

Der Roboter darf nicht überspringen, überfliegen, klettern, kratzen, schneiden, brennen, markieren, beschädigen oder zerstören der Wände des Labyrinths. IEEE, 2020, S.2 3.4

5. Größe der Micromouse:

Die Micromouse darf nicht länger oder breiter als 25 cm sein. Falls der Roboter in der Lage ist seine Form während des Betriebs zu ändern, dürfen die maximalen Abmessungen dabei immer noch nicht 25 cm × 25 cm überschreiten. In der Höhe gibt es keine Größenbegrenzungen. IEEE, 2020, S.2 3.5

6. Inspektionen:

Alle Roboter werden vor dem Start des Wettbewerbs überprüft. Es wird sichergestellt dass sie der Regelkonformität entsprechen und dass sie keine potenzielle Sicherheitsrisiken darstellen. IEEE, 2020, S.2 3.6

7. Regelverstöße:

Jeglicher Regelverstoß wird mit sofortiger Disqualifikation vom Wettbewerb geahndet. IEEE, 2020, S.2 3.7

Labyrinth-Spezifikationen

1. Labyrinthmaße:

Das Labyrinth besteht aus 18 cm × 18 cm großen quadratischen Einheiten, die zu einem 16 × 16 Gitter angeordnet sind. Die Wände sind 5 cm hoch und 1,2 cm dick. Eine Toleranz von 5 % Abweichung in den Maßen der Wände ist zulässig. Das gesamte Labyrinth wird von einer Außenwand umschlossen, die den Rand des Gitters bildet. IEEE, 2020, S.2 4.1

2. Farben im Labyrinth:

Die Seiten der Wände des Labyrinths sind weiß, die Oberseite ist rot. Der Boden ist aus in schwarz gefärbtem Holz. Das Labyrinth kann Fehler beinhalten. Man sollte nicht davon ausgehen, dass alles perfekt ist. Die Holzbeschaffenheit kann variieren, zwischen den aneinandergelegten Holzplatten können kleine Spalten oder Kanten sein. Die Farbe kann von der Deckkraft oder Sättigung inkonsistent sein. IEEE, 2020, S.3 4.2

3. Start-/ Endzonen:

Das Startquadrat des Labyrinths liegt in einer der vier Ecken. Der Start ist mit drei Wänden umrandet. Die Startlinie liegt zwischen dem ersten und dem zweiten Quadrat. Sobald die Micromouse die Startlinie überschreitet, startet der Timer des Durchlaufs. Das Ziel ist der Eingang zu den vier Einheitsquadraten in der Mitte des Labyrinths. Das Ziel hat nur einen Eingang. IEEE, 2020, S.3 4.3

4. **Quadratische Pfosten:**

In den vier Ecken des Einheitquadrates befindet sich ein quadratischer Pfosten mit den Maßen $1,2\text{ cm} \times 1,2\text{ cm}$. Das Labyrinth ist so aufgebaut, dass an jedem dieser Pfosten mindestens eine Wand anliegt. IEEE, 2020, S.3 4.4

5. **Mehrzahl an Wegen:**

Es sind mehrere Wege vom Start bis zum Ende erlaubt und möglich. Das Zielquadrat wird so positioniert, dass eine Wändeabtastmaus das Ziel nicht finden kann. IEEE, 2020, S.3 4.5

Wettbewerbsregeln

1. **Zeit:**

Jedes Team hat 10 Minuten Zugang zum Labyrinth. Die Zeit startet, wenn der Wettbewerbsleiter das Team bestätigt, und den Zugang zu dem Labyrinth gewährt. Die Zeit zwischen den Durchläufen, die von den Teams beansprucht werden kann um den Roboter anzupassen, ist in diesen 10 Minuten enthalten. Eine Laufzeit wird für jeden Lauf (von der Startzelle bis zur Mittelzone), in dem der Roboter erfolgreich das Zielfeld erreicht, aufgezeichnet. Die geringste gemessene Zeit innerhalb der 10-Minütigen Versuchszeit ist die offizielle Zeit der Maus. IEEE, 2020, S.3 5.1

2. **Stoppen/ Entfernen des Micromouse Roboters:**

Jeder Durchlauf muss vom Startfeld aus erfolgen. Mehrere Läufe bzw. Laufversuche können innerhalb der zugewiesenen Zeit von 10 Minuten durchgeführt werden. Das Team kann einen Durchlauf jederzeit abbrechen und die Micromouse auf das Startfeld zurücksetzen. Alternativ kann er auch jederzeit entfernt werden, ohne dass sich dies auf die Laufzeit des Durchlaufs auswirkt. Wird der Roboter wieder im Labyrinth auf dem Startfeld platziert, wird eine einmalige Strafe von 30 Sekunden zur nächsten Laufzeit des Roboters aufaddiert. IEEE, 2020, S.4 Kap. 5.2

3. **Neuprogrammierung nach der Enthüllung:**

Nachdem den Teams zu Beginn des Wettbewerbs das Labyrinth gezeigt wurde, darf der Bediener den Micromouseroboter nicht umprogrammieren. Er kann jedoch Schalter, die auf dem Roboter platziert sind, ändern. IEEE, 2020, S.4 Kap.5.3

4. **Raumverhältnisse:**

Beleuchtungsstärke, Temperatur und Luftfeuchtigkeit des Raums müssen denen einer normalen Umgebung entsprechen. (40 bis 120 °F, 0 % to 95 % Luftfeuchtigkeit, nicht kondensierend) IEEE, 2020, S.4 Kap.5.5

5. **Laufzeit:**

Die Stoppuhr startet, wenn die Vorderkante des Micromouseroboters die Startlinie überquert, und stoppt, wenn die Vorderkante der Maus die Ziellinie überquert. Die Startlinie befindet sich an der Grenze zwischen dem Startquadrat und dem nächsten Quadrat im Uhrzeigersinn. Die Ziellinie befindet sich am Eingang des Zielquadrats. IEEE, 2020, S.4 Kap.5.5

6. **Starten eines Laufs:**

Jedes Mal, wenn der Micromouse-Roboter das Startfeld verlässt, beginnt ein neuer Lauf. Wenn der Roboter das Zielfeld nicht betritt, wird keine Laufzeit aufgezeichnet. Wenn der Roboter beispielsweise während eines Laufs das Startfeld erneut betritt (bevor er das Zielfeld erreicht), wird dieser Lauf abgebrochen und eine neue Laufzeit beginnt, sobald der Roboter das Startfeld verlässt. IEEE, 2020, S.4 Kap.5.6

7. **Fortführende Navigation:**

Wenn der Micromouse-Roboter nach Erreichen des Zielfeldes weiter durch das Labyrinth navigiert, wird die dafür benötigte Zeit nicht für einen Durchlauf angerechnet. Der Roboter kann und sollte mehrere Durchläufe machen, ohne vom Bediener berührt zu werden. Wenn der Roboter die Mitte gefunden hat, ist es üblich, das Labyrinth auf dem Rückweg zum Startplatz über einen alternativen Weg zu erkunden. IEEE, 2020, S. 4f Kap.5.7

8. **Modifizierung des Roboters:**

Das Team darf keine Informationen über das Labyrinth an den Micromouse-Roboter weitergeben. Daher ist es nicht erlaubt, ROMs zu ändern oder Programme herunterzuladen, sobald das Labyrinth enthüllt ist. Die Teilnehmer dürfen jedoch Folgendes tun:

- Schaltereinstellungen ändern (z.B. zur Auswahl von Algorithmen).
- Batterien zwischen den Läufen austauschen.
- Sensoren einstellen.
- Geschwindigkeitseinstellungen ändern.
- Reparaturen durchführen.

IEEE, 2020, S.5 Kap.5.8

9. **Gewicht des Roboters ändern:**

Nachdem die 10 Minuten für das Team gestartet sind, darf der Micromouse-Roboter nicht so verändert werden, dass sich das Gewicht ändert (z. B. ist es nicht erlaubt, eine sperrige Sensoranordnung zu entfernen oder auf leichtere Batterien umzusteigen, um nach der Kartierung des Labyrinths eine höhere Geschwindigkeit zu erzielen). Die Schiedsrichter entscheiden über alle Interaktionen mit dem Roboter. IEEE, 2020, S.5 Kap.5.9

Punktevergabe

1. Regionalität und Präsentation:

Es gibt nur einen offiziellen IEEE Micromousewettbewerb pro Jahr in jedem Gebiet oder Region. Alle Micromouseroboter, unabhängig davon, ob sie an früheren Wettbewerben teilgenommen haben oder nicht, konkurrieren auf der gleichen Basis. Alle Roboter müssen vom original Team den Richtern präsentiert werden. IEEE, 2020, S.5 Kap.6.1

2. Punktevergabe:

Der erste Platz geht an das Team mit der kürzesten offiziellen Zeit (ohne dass das Team seinen Micromouse-Roboter während der Läufe berührt). Der zweite Preis geht an das Team mit der nächstkürzeren Zeit usw. Teams, deren Micromouse-Roboter nicht in die Mitte des Platzes einfährt, werden von den Juroren nach folgenden Kriterien eingestuft

- Wie nahe der Roboter dem Zielfeld kommt, ohne berührt zu werden.
- Nachweis, dass die Maus weiß, wo sie sich in Bezug auf das Zielfeld befindet.

Wenn der Roboter gelegentlich in einer Ecke oder an einer Wand stecken bleibt, kann das Team manuell eingreifen, um das Problem zu beheben (wobei darauf zu achten ist, dass die beabsichtigte Bewegungsrichtung der Maus nicht verändert wird). Die Häufigkeit solcher Korrekturen wird von den Schiedsrichtern bei der Wertung berücksichtigt. IEEE, 2020, S.5 Kap.6.2

3. Beantragung von Pausen:

Auf Wunsch kann dem Team nach Beendigung eines Laufs eine Pause gewährt werden, wenn noch ein anderes Team auf den Antritt wartet. Der 10-Minuten-Timer wird angehalten. Die Schiedsrichter und Verantwortlichen entscheiden von Fall zu Fall, ob eine Pause erlaubt ist. IEEE, 2020, S.6 Kap.6.3

4. Ermessensspielraum der Richter:

Die Juroren behalten sich das Recht vor, das Team um eine Erklärung der Micromouse und ihrer Aktionen zu bitten. Die Juroren behalten sich auch das Recht vor, einen Lauf zu stoppen, eine Disqualifikation auszusprechen oder Anweisungen zu erteilen (z.B. wenn die Struktur des Labyrinths durch den weiteren Betrieb der Maus gefährdet ist). IEEE, 2020, S.6 Kap.6.4

Ausgehend von den Regeln darf mit dem in der Studienarbeit behandelten Roboter Pololu 3Pi+ 2040 an keinem offiziellen Wettbewerb teilnehmen. Das war vor der Studienarbeit schon bewusst und wurde auch noch nie angenommen. Der Roboter kann jedoch sehr nah an einem Wettbewerb angelehnt betrieben werden, da es bei der Micromouse auch stark auf die Software ankommt. Die Hardware erfüllt die Bedingungen, basiert aber auf einem fertigen Produkt, das nicht von Null aus gebaut wurde. Die Rahmenbedingungen sind stark an die offiziellen Regeln angelehnt, es obliegt dem Anwender, wie die Micromouse bedient werden soll, und was damit angestellt werden soll. Die Studienarbeit fokussiert sich stärker auf das Thema, wie der Roboter aufgebaut ist, und wie man ihn ansteuern kann und wie jemand, der noch nie von einer Micromouse gehört hat, in das Thema eingeleitet wird, um selber ein solches Projekt zu verwirklichen. Aus diesem Grund wurde die Bodenplatte auch nicht schwarz gefärbt, wie es in den Regeln steht. So besteht die

Möglichkeit, die Grundplatte noch anderweitig zu verwenden. Es kann beispielsweise auch mit Tape ein Labyrinth geklebt werden, um so ein temporäres Labyrinth zu fertigen, das nicht auf den Wänden beruht. So sind die Einsatzmöglichkeiten der Hardware einfach größer. Da der Grundgedanke der Studienarbeit stark an der Micromouse angelehnt ist, wird die Information über die Regeln als wichtiger Bestandteil der Arbeit erachtet.

4.2 Aufbau und Fertigung des Labyrinths

Das Labyrinth besteht aus modularen Einheiten, die je nach Bedarf flexibel zusammengesetzt werden können. Die Bodenplatten sind aus mitteldichten Holzfaserplatten (MDF) gefertigt. MDF bietet mehrere Vorteile: Es lässt sich leicht mechanisch bearbeiten, ist kostengünstig (12 mm: ca. 25,30 €/m² Holzplatte, [2025a](#)) und weist im Vergleich zu anderen Holzwerkstoffen wie Multiplex (12 mm: ca. 61,40 €/m² Holzplatte, [2025b](#)) oder Siebdruckplatten (12 mm: ca. 77,30 €/m² Holzplatte, [2025c](#)) eine gute Verfügbarkeit sowie eine hohe geometrische Formstabilität auf. Zudem besitzt es eine glatte Oberfläche, die sich besonders gut für den Einsatz mit kleinen Robotern eignet, im Gegensatz zu beispielsweise OSB-Platten (12 mm: ca. 26,20 €/m² Holzplatte, [2025d](#)), die eine gröbere Struktur aufweisen.

Die genannten Preise stellen lediglich eine Momentaufnahme dar, da sich Holzpreise laufend ändern und je nach Anbieter variieren können. Sie dienen ausschließlich zur Veranschaulichung der preislichen Unterschiede.

Da die Platten im Betrieb keinen hohen mechanischen Belastungen ausgesetzt sind, stellen sie eine geeignete Wahl als Grundplatten dar. Eine wesentliche Anforderung an das Material ist seine Formstabilität über die Zeit, insbesondere um Verwindungen oder Verformungen zu vermeiden. Eine wellige oder unebene Oberfläche könnte das Fahrverhalten der Roboter beeinträchtigen oder sogar dazu führen, dass sie stecken bleiben, da sie auf eine ebene Fläche ausgelegt sind. Zudem könnte sich bei einer ungleichmäßigen Oberfläche die Passgenauigkeit der Wandmodule verschlechtern, da deren Position durch feste Lochabstände bestimmt wird. Eine hohe Präzision der Bohrungen ist daher essenziell, aus diesem Grund wurden die Platten extern von einem Fachunternehmen gebohrt. Der Bohrplan befindet sich im Anhang B und wurde von Herrn Cabrera-Schmitt geplant und gezeichnet.

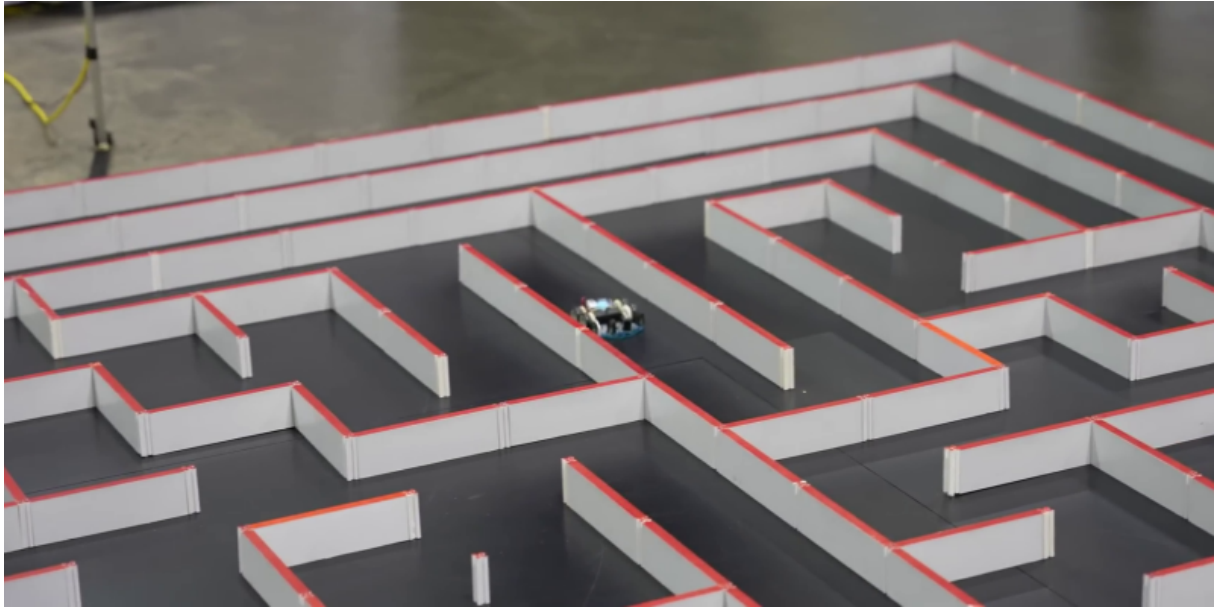


Abbildung 15: Labyrinth eines offiziellen Micromouse-Wettbewerbs
Conradie, [2023](#)

4.3 3D-Drucker als Medium zur Realisierung der Kunststoffkomponenten

Im Rahmen der Studienarbeit wurden die Wände und Eckpfeiler des Modells mit zwei unterschiedlichen 3D-Druckern gefertigt: dem *Prusa Mk3s+* sowie dem *Anycubic i3 Mega S*. Der Einsatz additiver Fertigungsverfahren ermöglichte die effiziente und kostengünstige Herstellung individueller Bauteile. Insbesondere bei iterativen Entwicklungsprozessen bietet der 3D-Druck den Vorteil, dass Konstruktions- oder Parametrierungsfehler bereits in frühen Stadien durch Probedrucke erkannt und behoben werden können. Aufwendige und teure Sonderanfertigungen entfallen dadurch.

Durch die flexible Anpassung von Druckparametern lassen sich auch experimentelle Modifikationen unkompliziert umsetzen. Allerdings ist das Ergebnis stark von der Wahl dieser Parameter abhängig. Eine ungeeignete Konfiguration kann zu erheblichen Qualitätsverlusten führen. Es ist daher eine gewisse Erfahrung erforderlich, um die Einstellungen optimal auf das jeweilige DruckermodeLL abzustimmen. Sobald jedoch ein zufriedenstellender Satz an Parametern gefunden und das 3D-Modell in eine G-Code-Datei umgewandelt wurde (dieser Vorgang wird als „Slicing“ bezeichnet), lässt sich der Druckvorgang in der Regel problemlos wiederholen – vorausgesetzt, das verwendete Material bleibt unverändert.

Als Druckmaterial kam weißes Polylactid (PLA) der Firma Prusa zum Einsatz. Dieses Material zeichnet sich durch eine einfache Verarbeitung sowie geringe Kosten aus. Ein wesentlicher Vorteil des 3D-Drucks liegt in der unmittelbaren Kontrolle über den Produktionsprozess, wodurch Anpassungen jederzeit vorgenommen werden können.

Die Bauteile basieren auf einem Feder-Nut-System, bei dem die Passgenauigkeit der Elemente eine zentrale Rolle spielt. Sind die Toleranzen zu groß, besteht die Gefahr, dass die Feder aus der Nut herausrutscht. Sind sie hingegen zu gering, kann es zu einem Verhaken oder vollständigen Blockieren der Verbindung kommen, was die Gefahr mechanischer Beschädigungen birgt. Ziel ist es daher, eine optimale Toleranz zu erreichen, die ein leichtes und zugleich sicheres Ineinanderstecken der Komponenten erlaubt. Die 3D-Modelle befinden sich im Anhang I. jbabio, [2017](#)

Das eingesetzte Fertigungsverfahren basiert auf dem Fused Deposition Modeling (FDM). Dabei wird ein thermoplastischer Kunststoff – im vorliegenden Fall PLA – in Form eines Filaments durch eine beheizte Düse extrudiert und schichtweise aufgetragen. Das eingesetzte Filament besitzt einen Durchmesser von 1,75 mm. Das zugrunde liegende Prinzip beruht auf der digitalen Zerlegung des 3D-Modells in einzelne Layer. Für die vorliegenden Drucke wurde eine Schichthöhe von 0,2 mm gewählt. Je geringer die gewählte Schichthöhe, desto feiner können Details im Modell wiedergegeben werden. Schebesta, 2017, S.12f

Fehlerhafte Einstellungen in diesem Prozess können zu suboptimalen Ergebnissen oder sogar zum vollständigen Fehlschlag des Druckvorgangs führen. Insbesondere die Maßhaltigkeit ist bei passgenauen Systemen, wie dem eingesetzten Feder-Nut-Prinzip, von zentraler Bedeutung. Abweichungen durch unzureichende Parametrierung können dazu führen, dass Bauteile nicht korrekt zusammengefügt werden können. Obwohl eine perfekte Reproduzierbarkeit kaum erreichbar ist, sollte dennoch ein hoher Qualitätsstandard angestrebt werden, um funktionsfähige und stabile Komponenten zu erhalten.

Im Rahmen dieser Arbeit wurden zwei unterschiedliche Slicing-Programme verwendet: *PrusaSlicer* für den Prusa Mk3s+ und *Ultimaker Cura* für den Anycubic i3 Mega S. Da sich beide Programme im grundlegenden Funktionsprinzip ähneln und hauptsächlich in ihrer Benutzeroberfläche unterscheiden, wird im Folgenden ausschließlich auf die Arbeit mit *Ultimaker Cura* eingegangen. Dies erfolgt unter anderem aufgrund der umfangreicheren praktischen Erfahrung mit dieser Software, die in den Jahren zuvor erlangt wurde.

Wesentliche Einflussgrößen sind dabei Druckgeschwindigkeit, Materialverbrauch, mechanische Stabilität und Oberflächenqualität. Diese Parameter stehen jedoch in einem Zielkonflikt zueinander: Eine Erhöhung der Druckgeschwindigkeit geht in der Regel zulasten der Druckqualität und der strukturellen Festigkeit. Umgekehrt erfordern qualitativ hochwertige und besonders stabile Bauteile einen erhöhten Materialeinsatz sowie längere Fertigungszeiten. Eine simultane Optimierung aller genannten Faktoren ist daher kaum möglich, weshalb stets eine anwendungsbezogene Priorisierung erfolgen muss.

Zusätzlich zu den systeminternen Parametern wirken externe Faktoren wie Materialeigenschaften, Umgebungstemperatur und Luftfeuchtigkeit auf das Druckergebnis ein. Selbst Filamente mit identischer Bezeichnung können chargenbedingt variieren. Unterschiede im Herstellungsprozess, Alter oder Lagerbedingungen des Materials können zu abweichendem Druckverhalten führen. Insbesondere bei längerer oder unsachgemäßer Lagerung kann Feuchtigkeit in das Filament eindringen, was die Druckqualität erheblich beeinträchtigt.

Zur Bestimmung optimaler Druckbedingungen werden vor dem finalen Druck in der Regel Testdrucke durchgeführt. Hierbei kommen häufig sogenannte Temperaturtürme (*Temptower*) zum Einsatz, mit deren Hilfe die geeignete Extrusionstemperatur ermittelt werden kann. Ein Temperaturturm besteht aus mehreren Segmenten, die mit jeweils unterschiedlichen Düsentemperaturen gedruckt werden – beginnend mit dem höchsten Temperaturwert im unteren Bereich bis hin zur niedrigsten Temperatur im oberen Abschnitt. Anhand der sichtbaren Druckqualität der einzelnen Segmente lässt sich diejenige Temperatur identifizieren, bei der das Filament die besten Ergebnisse liefert. Diese wird anschließend für den eigentlichen Druckvorgang übernommen.

Ein weiterer zentraler Parameter im FDM-Verfahren ist der Durchmesser der verwendeten Düse, da er maßgeblich die Druckgeschwindigkeit, Präzision und Stabilität des Bauteils beeinflusst. Im Labor von Prof. Dr.-Ing. Alexander Biedermann, in dem der *Prusa Mk3s+* betrieben wird, ist standardmäßig eine Düse mit einem Durchmesser von 0,4 mm verbaut.

Beim *Anycubic i3 Mega S* hingegen wird eine Düse mit 0,6 mm verwendet. Ein Austausch der Düse ist grundsätzlich möglich, erfordert jedoch zusätzlichen Aufwand und birgt insbesondere bei öffentlich zugänglichen Geräten die Gefahr fehlerhafter Ausdrücke, wenn andere Nutzer nicht über die geänderte Konfiguration informiert sind.

Kleinere Düsen ermöglichen eine höhere Detailgenauigkeit, da die minimale Linienbreite dem Düsendurchmesser entspricht. Mit einer 0,4 mm-Düse können somit präzisere Konturen erzeugt werden. Allerdings ist die Druckgeschwindigkeit geringer, da weniger Material pro Zeiteinheit extrudiert werden kann. Größere Düsen, wie die mit 0,6 mm, erlauben einen schnelleren Materialauftrag und führen zu stabileren Bauteilen, sind jedoch hinsichtlich der Detailwiedergabe limitiert.

Im Kontext dieser Arbeit bezieht sich der Begriff „Wand“ nicht auf das gesamte Bauteil (z.B. die Wände eines Labyrinths), sondern ausschließlich auf die äußere Hülle des gedruckten Objekts. Diese sogenannte Außenwand besteht aus mehreren Linienzügen, deren Breite der eingestellten Linienbreite entspricht, welche wiederum vom Düsendurchmesser abhängt. Für eine gute Druckqualität sollte die Wandstärke ein Vielfaches der Linienbreite betragen. Beträgt beispielsweise die gewünschte Wandstärke 1,2 mm, so werden bei einer 0,4 mm-Düse drei Linien benötigt, während eine 0,6 mm-Düse dieselbe Wandstärke bereits mit zwei Linien realisieren kann.

Je nach Modellgeometrie und dem Anteil der Wandflächen an der gesamten Struktur kann somit durch den Einsatz einer größeren Düse die Druckzeit signifikant reduziert werden. Insbesondere bei umfangreichen Druckaufträgen mit hoher Wandanteilfläche kann dies zu einer deutlichen Zeitersparnis führen, ohne dabei die strukturelle Integrität zu gefährden.

Da die Bauteile im Betrieb der Micromouse lediglich geringen mechanischen Belastungen in Form von Stößen ausgesetzt sind und keine tragenden Funktionen erfüllen, ist eine besonders stabile Druckweise, beispielsweise mit 100 % Füllung, nicht erforderlich. Der Fokus liegt vielmehr auf einer hohen Maßhaltigkeit und Oberflächenqualität, da das verwendete Feder-Nut-System präzise passen muss.

Um Material zu sparen und die Druckzeit zu verkürzen, wurde bewusst mit 0 % Infill gearbeitet. Die Stabilität der Außenwände reicht in Anbetracht der geringen mechanischen Belastung aus, um dennoch funktionsfähige Bauteile zu gewährleisten.

Die gewählte Druckgeschwindigkeit betrug 50 mm/s, was einen guten Kompromiss zwischen Qualität und Druckdauer darstellt. Zwar ließe sich vermuten, dass insbesondere die geradlinigen Strukturen der Labyrinthwände schneller gedruckt werden könnten, jedoch würde dabei auch die im gleichen Prozess gefertigte Feder mit erhöhter Geschwindigkeit gefertigt. Dies könnte zu Qualitätseinbußen führen, die die Funktionalität beeinträchtigen. Die potenzielle Zeitersparnis steht daher in keinem Verhältnis zu den möglichen Qualitätsverlusten – hier wurde bewusst eine konservativere Einstellung gewählt.

Der erste Layer wurde mit einer reduzierten Geschwindigkeit von 30 mm/s gedruckt, da dieser für den gesamten Druckprozess von entscheidender Bedeutung ist. Fehler im ersten Layer, etwa unzureichende Haftung oder Ablösungen, können zu Schichtverschiebungen, Verformungen oder im schlimmsten Fall zu einem vollständigen Druckversagen führen. Solche Fehler können nicht nur das Bauteil unbrauchbar machen, sondern auch den Drucker beschädigen. Eine langsamere Druckgeschwindigkeit beim ersten Layer minimiert dieses Risiko, auch wenn sie keine absolute Fehlerfreiheit garantiert. Eine genauere Betrachtung dieser Problematik erfolgt im Fazit.

Für die Drucktemperatur wurde ein Wert von 205 °C gewählt. Dieser Parameter erwies sich in vorangegangenen Tests mit Temperatortürmen als optimal. Leider liegt kein Bildmaterial der Temperatortürme vor, da diese im Rahmen von Stabilitätstests gezielt zerstört wurden, um die mechanische Belastbarkeit der Drucke bei unterschiedlichen Temperaturen zu überprüfen. Anschließend wurden die Testkörper entsorgt.

Die Bauteile wurden mit maximaler Bauteilkühlung gedruckt, was bei der Verwendung von PLA empfehlenswert ist, um saubere Druckergebnisse zu erzielen. Zudem kam ein Retract von 4 mm zum Einsatz. Dieser sorgt dafür, dass das Filament während Bewegungen des Druckkopfs ohne Extrusion zurückgezogen wird, um das ungewollte Austreten von Material („Oozing“) zu minimieren und Fädenbildung zu verhindern.

Auf eine detaillierte Erläuterung sämtlicher Druckparameter wird an dieser Stelle verzichtet, da dies den Rahmen sprengen würde. Im Folgenden ist jedoch eine Tabelle enthalten, die sämtliche verwendeten Einstellungen zusammenfasst. Bei weiterführendem Interesse bietet die Slicing-Software Cura selbst Erklärungen zu den einzelnen Parametern: Fährt man mit dem Mauszeiger über die jeweilige Einstellung, wird eine kurze Beschreibung eingeblendet. Voraussetzung hierfür ist die Verwendung einer aktuellen Version von Cura.

Tabelle 6: Ausgewählte Druckparameter

Parameter	Einstellung
Druckverfahren	Fused Deposition Modeling (FDM)
Filament	PLA, 1,75 mm
Düsentemperatur	205 °C
Schichthöhe	0,2 mm
Düsendurchmesser	0,4 mm (Prusa), 0,6 mm (Anycubic)
Druckgeschwindigkeit	50 mm/s
Geschwindigkeit erster Layer	30 mm/s
Infill	0 %
Retract	4 mm
Bauteilkühlung	maximal

5 Fernziele

Die vorliegende Studienarbeit dokumentiert die erfolgreiche Inbetriebnahme des Micromouse-Roboters Pololu 3Pi+ 2040. Es wurde detailliert dargelegt, wie die Hardwarekomponenten montiert, die umfangreiche Sensorik angesteuert und die Aktoren mittels der Arduino-IDE gesteuert werden. Die Arbeit zeigt grundlegende Herausforderungen in der autonomen Robotik auf, wie die Kalibrierung von Sensoren und die Notwendigkeit präziser Motorsteuerung.

Der Roboter dient in seiner jetzigen Form als eine wertvolle Lernplattform, die es Studierenden ermöglicht, sich mit den Kernkonzepten der Labyrinth-Navigation und der Sensordatenverarbeitung vertraut zu machen. Obwohl das verwendete Modell aufgrund der Wettbewerbsregeln nicht an offiziellen Wettkämpfen teilnehmen kann, bildet die in dieser Arbeit geschaffene Wissensbasis ein solides Fundament für die zukünftige Entwicklung eines vollständig eigenkonstruierten Roboters.

Für Projekte mit einem eng bemessenen Zeitrahmen stellt die Implementierung eines Labyrinths auf Basis von Bodenmarkierungen eine zielführende Alternative dar. Die Verwendung von Liniensensoren ist mit geringeren technischen Herausforderungen verbunden als die Navigation mittels Distanzsensoren, was eine schnellere Realisierung eines funktionsfähigen Systems ermöglicht. Die Wahl der Methode sollte sich jedoch stets nach dem übergeordneten Ziel der Aufgabenstellung richten.

Aufbauend auf den Ergebnissen dieser Arbeit ergeben sich mehrere vielversprechende Ansatzpunkte für zukünftige Optimierungen und Erweiterungen des Projekts.

Im Rahmen der Studienarbeit gäbe es weitere Dinge, die verbessert werden könnten. Eine Verbesserung wäre, nicht nur den ToF, sondern den ganzen Adapter steckbar zu machen. So könnte man den Roboter noch modularer aufbauen, da die Pins zur freien Verfügung stünden. Durch die Vertauschung der Pins durch den Adapter ist es nicht mehr eindeutig erkennbar, welcher Pin welchem Anschluss zugeordnet ist. Das stellt eine Fehlerquelle für potenzielle zukünftige Aufbauten dar.

Darüber hinaus wäre die Evaluation alternativer Sensortechnologien zur Distanzmessung, wie beispielsweise Ultraschallsensoren, ein interessanter nächster Schritt. Diese wären unempfindlich gegenüber den Lichtverhältnissen der Umgebung, würden jedoch neue mechanische und softwareseitige Herausforderungen mit sich bringen, etwa durch die Notwendigkeit mehrerer Sensoren oder eines rotierenden Systems zur 180°-Abdeckung.

Die Integration eines WLAN- oder Bluetooth-Moduls würde das Einsatzspektrum des Roboters erheblich erweitern. Rechenintensive Aufgaben, wie die Implementierung selbstlernender Algorithmen oder komplexer Regelungsstrategien, könnten auf einen leistungsstarken externen PC ausgelagert werden. Der Roboter würde dann lediglich als Akteur fungieren, der die berechneten Steuersignale umsetzt. Dies öffnet die Tür für anspruchsvolle Projekte im Bereich des maschinellen Lernens.

Ein weiterer Ansatzpunkt für zukünftige Arbeiten liegt im Bereich des Energiemanagements. Hierbei könnte eine detaillierte Analyse des Energieverbrauchs der einzelnen Komponenten erfolgen, mit dem Ziel, Strategien zur Maximierung der Akkulaufzeit zu entwickeln. Die Optimierungspotenziale liegen dabei sowohl in der softwareseitigen Effizienz durch ressourcenschonenden Code, als auch in der algorithmischen Effizienz, die durch eine schnelle Labyrinthlösung die Gesamtfahrzeit und somit den Energiebedarf minimiert.

Literatur

- Aschaffenburg, T. H. (2024). *Labor für Robotics and Internet of Things*. Verfügbar 15. Dezember 2024 unter <https://www.th-ab.de/hochschule/organisation/organisationseinheiten/labor-robotics-and-internet-of-things>
- Bielefeld, H. (2024). *Robotik*. Verfügbar 15. Dezember 2024 unter <https://www.hsbi.de/iium/studium/labore/laborbereich-physik/-labor-lasertechnik-und-optoelektronik/robotik>
- Christiansen, D. (2014). *The Amazing MicroMouse Contest*. Verfügbar 15. Dezember 2024 unter <https://spectrum.ieee.org/the-amazing-micromouse-contest>
- Conradie, D. (2023). Verfügbar 4. Februar 2025 unter <https://hackaday.com/2023/05/28/the-fascinating-evolution-of-micromouse/>
- COX, P., & DRY, G. (2003). *RICHTLINIE 2002/95/EG DES EUROPÄISCHEN PARLAMENTS UND DES RATES vom 27. Januar 2003 zur Beschränkung der Verwendung bestimmter gefährlicher Stoffe in Elektro- und Elektronikgeräten*. Verfügbar 15. Dezember 2024 unter <https://eur-lex.europa.eu/legal-content/DE/TXT/PDF/?uri=CELEX:32002L0095>
- FMH-Studios. (2020). *I2C-Bus*. Verfügbar 22. Mai 2025 unter <https://fmh-studios.de/theorie/informationstechnik/i2c-bus/>
- Grayson, D., Grayson, P., & Kevin. (2021). *pololu 3pi plus 2040 arduino library*. Verfügbar 23. Mai 2025 unter <https://github.com/pololu/pololu-oled-arduino>
- Grayson, D., Mulligan, R., & Kevin. (2020). *opt3101-arduino*. Verfügbar 22. Mai 2025 unter <https://github.com/pololu/opt3101-arduino>
- Green, A. (2024). *pololu 3pi plus 2040 arduino library*. Verfügbar 21. Februar 2025 unter <https://github.com/adamgreen/pololu-3pi-plus-2040-arduino-library>
- Holzplatte. (2025a). Verfügbar 4. Februar 2025 unter <https://holzplatte.de/holzzuschnitt/mdf/112/mdf-platte-8-mm-12-mm-16-mm-19-mm-22-mm-25-mm>
- Holzplatte. (2025b). Verfügbar 4. Februar 2025 unter <https://holzplatte.de/holzzuschnitt/multiplex-birke/109/multiplex-birke-4-mm-9-mm-12-mm-15-mm-18-mm-21-mm>
- Holzplatte. (2025c). Verfügbar 4. Februar 2025 unter <https://holzplatte.de/holzzuschnitt/siebdruckplatten/111/siebdruck-platte-9-mm-12-mm-15-mm-18-mm-21-mm-24-mm-27-mm>
- Holzplatte. (2025d). Verfügbar 4. Februar 2025 unter <https://holzplatte.de/holzzuschnitt/osb/110/osb-platte-12-mm-15-mm-18-mm-22-mm-25-mm>
- IEEE. (2020). *MicroMouse Competition Rules*. Verfügbar 22. Januar 2025 unter https://attend.ieee.org/r2sac-2020/wp-content/uploads/sites/175/2020/01/MicroMouse_Rules_2020.pdf
- jbbabio. (2017). *3D printable Micromouse labyrinth Walls and Connectors*. Verfügbar 23. Mai 2025 unter <https://www.thingiverse.com/thing:2594629/files>
- Krüger, M. (2008). *PDM-Signal mit einem Tastgrad*. Verfügbar 9. Dezember 2024 unter https://de.wikipedia.org/wiki/Pulsdauermodulation#/media/Datei:Pulse_wide_wave.svg
- Mankar, J., Darode, C., Trivedi, K., Kanoje, M., & Shahare, P. (2014). Review of I2C protocol. *International Journal of Research in Advent Technology*, 2(1). Verfügbar 12. Februar 2025 unter <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=314537daa1f601f83044b25b68e2af6c8f331f3f>
- Pini, A. (2020). *Warum der Inter-Integrated Circuit (I2C)-Bus den Anschluss von ICs so einfach macht - und wie man ihn verwendet*. Verfügbar 22. Mai 2025 unter

- <https://www.digikey.de/de/articles/why-the-inter-integrated-circuit-bus-makes-connecting-ics-so-easy>
- Pololu. (2024a). *3-Channel Wide FOV Time-of-Flight Distance Sensor Using OPT3101*. Verfügbar 10. Dezember 2024 unter <https://www.pololu.com/product/3412>
- Pololu. (2024b). *30:1 Micro Metal Gearmotor MP 6V with Extended Motor Shaft*. Verfügbar 10. Dezember 2024 unter <https://www.pololu.com/product/2378>
- Pololu. (2024c). *Pololu 3pi+ 2040 User's Guide*. Verfügbar 10. Dezember 2024 unter <https://www.pololu.com/docs/0J86/all#1>
- Pololu. (2024d). *Pololu QTR Reflectance Sensor Application Note*. Verfügbar 10. Dezember 2024 unter <https://www.pololu.com/docs/0J13>
- Pololu. (2025a). *Graphical OLED Display*. Verfügbar 23. Mai 2025 unter <https://www.pololu.com/product/3761>
- Pololu. (2025b). *Pin assignments*. Verfügbar 25. Februar 2025 unter <https://www.pololu.com/docs/0J86/6.9>
- Reichelt. (2017). *Temperatur und Lötkolben richtig auswählen*. Verfügbar 22. Mai 2025 unter <https://www.reichelt.de/magazin/ratgeber/temperatur-und-loetkolben-richtig-auswaehlen/>
- Schebesta, S. (2017). *Einfluss ausgewählter Druckparameter auf Form und Stabilität beim FDM-Verfahren* [Diss., Wien]. Verfügbar 12. Februar 2025 unter https://web.archive.org/web/20220131055338id_/https://repositum.tuwien.at/bitstream/20.500.12708/9806/2/Schebesta%20Stefan%20-%202017%20-%20Einfluss%20ausgewaehlder%20Druckparameter%20auf%20Form%20und...pdf
- STMicroelectronics. (2015). *Digital output magnetic sensor: ultra-low-power, high-performance 3-axis magnetometer*. Verfügbar 7. Januar 2025 unter <https://www.pololu.com/file/0J1089/LIS3MDL.pdf>
- STMicroelectronics. (2019). *Datasheet - LSM6DSO - iNEMO inertial module: always-on 3D accelerometer and 3D gyroscope*. Verfügbar 10. Dezember 2024 unter <https://www.pololu.com/file/0J1899/lsm6dso.pdf>
- Valdez, J., & Becker, J. (2015). *Understanding the I2C bus*. Texas instruments. Verfügbar 12. Februar 2025 unter <https://git.ondrovo.com/gex/gex-thesis/raw/commit/6e2c9aa5d2b72ae99a8af4266952d8d355f77bde/references/ti-understanding-i2c--slva704.pdf>
- Wasserman. (2021). *Gauß (Einheit)*. Verfügbar 7. Januar 2025 unter [https://de.wikipedia.org/wiki/Gau%C3%9F_\(Einheit\)](https://de.wikipedia.org/wiki/Gau%C3%9F_(Einheit))

Abbildungsverzeichnis

1	Micromouse	1
2	Pinout der Platine des Roboters	6
3	Darstellung einer Pulsweitenmodulation	9
4	Schaltplan eines QTR-Sensorkanals in RC-Beschaltung mit dimmbarer Emitter-LED.	12
5	Ausgangssignal des QTR-RC-Sensors (gelb) und die zugehörige Zeitmessung des Mikrocontrollers (blau) bei einem Abstand von 1/8 Zoll über einer weißen Oberfläche.	12
6	Ausgangssignal des QTR-1RC-Sensors (gelb) und die zugehörige Zeitmes- sung des Mikrocontrollers (blau) bei einem Abstand von 1/8 Zoll über einer weiß-schwarzen Grenzfläche.	12
7	Ausgangssignal des QTR-1RC-Sensors (gelb) und die zugehörige Zeitmes- sung des Mikrocontrollers (blau) bei einem Abstand von 1/8 Zoll über einer schwarzen Linie.	12
8	Richtung der feststellbaren Beschleunigung	15
9	Richtung der feststellbaren Drehrate	15
10	Magnetsensorachsen	16
11	OPT3101 - Channel FOV	17
12	Pololu 3pi+ 2040 B-und Resetbutton	21
13	Pfad zur Boardauswahl in der Arduino-IDE	22
14	Bibliothek als ZIP-Datei herunterladen	23
15	Labyrinth eines offiziellen Micromouse-Wettbewerbs	45
16	Bohrplan des Labyrinths	IV
17	Schaltplan des verwendeten Distanzsensors.	X

Tabellenverzeichnis

2	Technische Spezifikationen des Motors	10
3	Technische Spezifikationen des SH1106 OLED-Displays	18
4	Pinbelegung des OLED-Displays	18
5	Buzzer-Einstellmöglichkeiten für die Funktion play()	27
6	Ausgewählte Druckparameter	48
7	GPIO Pinout des Pololu 3pi+ 2040	I

Listings

1	Einfacher Blink-Sketch zur Funktionsüberprüfung	22
2	Einzelansteuerung der Motoren	24
3	Einzelansteuerung der Motoren	24
4	Ansteuerung der LEDs	25
5	Initialisierung der gelben LED	25
6	Buzzer-Ansteuerung mit PlayFrequency	26
7	Buzzer Ansteuerung, Green, 2024	26
8	Buzzer Sequenz mit Überprüfung,	27
9	Werteausgabe der Counts	28
10	Funktionen für den Stoßsensor,	28
11	Funktionen für den IMU-Sensor, Green, 2024	30
12	Funktionen und Variablen der Klasse für den ToF-Sensor, Green, 2024	33
13	Dynamische Zähler-Ausgabe auf dem OLED-Display	36
14	Taster einbinden	37
15	Serielle Kommunikation mit dem Arduino	38

A GPIO-Pinout des Pololu 3pi+ 2040

Tabelle 7: GPIO Pinout des Pololu 3pi+ 2040

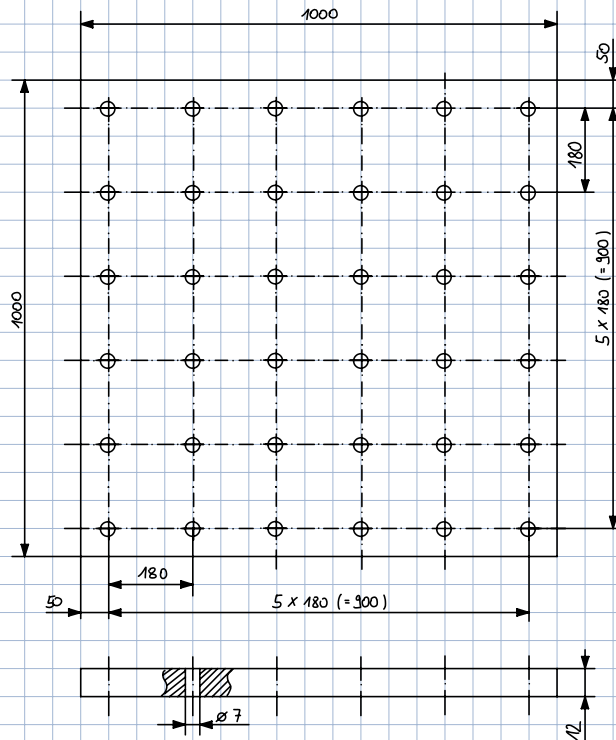
Pin	Alternative RP2040-Funktionen	Standardfunktion am 3pi+	Header
GP0	SPI0 RX, UART0 TX, I2C0 SDA, PWM0 A	Button C, Display data/command select (D/C)	Display
GP1	SPI0 CSn, UART RX, I2C0 SCL, PWM0 B	Display reset (Reset)	Display
GP2	SPI0 SCK, UART CTS, I2C1 SDA, PWM1 A	Display clock (SCK)	Display
GP3	SPI0 TX, UART RTS, I2C1 SCL, PWM1 B	Display data (MOSI), RGB LED data	Display, RGB (indirekt)
GP4	SPI0 RX, UART1 TX, I2C0 SDA, PWM2 A	Inertial sensors SDA	Mid expansion
GP5	SPI0 CSn, UART1 RX, I2C0 SCL, PWM2 B	Inertial sensors SCL	Mid expansion
GP6	SPI0 SCK, UART1 CTS, I2C1 SDA, PWM3 A	RGB LED clock	Mid expansion, RGB (indirekt)
GP7	SPI0 TX, UART1 RTS, I2C1 SCL, PWM3 B	Buzzer PWM	Mid expansion
GP8	SPI1 RX, UART1 TX, I2C0 SDA, PWM4 A	Right encoder A	—
GP9	SPI1 CSn, UART1 RX, I2C0 SCL, PWM4 B	Right encoder B	—
GP10	SPI1 SCK, UART1 CTS, I2C1 SDA, PWM5 A	Right motor direction	—
GP11	SPI1 TX, UART1 RTS, I2C1 SCL, PWM5 B	Left motor direction	—
GP12	SPI1 RX, UART0 TX, I2C0 SDA, PWM6 A	Left encoder A	—
GP13	SPI1 CSn, UART0 RX, I2C0 SCL, PWM6 B	Left encoder B	—
GP14	SPI1 SCK, UART0 CTS, I2C1 SDA, PWM7 A	Right motor PWM	—

Pin	Alternative RP2040-Funktionen	Standardfunktion am 3pi+	Header
GP15	SPI1 TX, UART0 RTS, I2C1 SCL, PWM7 B	Left motor PWM	—
GP16	SPI0 RX, UART0 TX, I2C0 SDA, PWM0 A	Right bump sensor (BR)	Front expansion
GP17	SPI0 CSn, UART0 RX, I2C0 SCL, PWM0 B	Left bump sensor (BL)	Front expansion
GP18	SPI0 SCK, UART0 CTS, I2C1 SDA, PWM1 A	Line sensor 5 (DN5, rightmost)	Front expansion
GP19	SPI0 TX, UART0 RTS, I2C1 SCL, PWM1 B	Line sensor 4 (DN4)	Front expansion
GP20	SPI0 RX, UART1 TX, I2C0 SDA, PWM2 A	Line sensor 3 (DN3)	Front expansion
GP21	SPI0 CSn, UART1 RX, I2C0 SCL, PWM2 B	Line sensor 2 (DN2)	Front expansion
GP22	SPI0 SCK, UART1 CTS, I2C1 SDA, PWM3 A	Line sensor 1 (DN1, leftmost)	Front expansion
GP23	SPI0 TX, UART1 RTS, I2C1 SCL, PWM3 B	Bump sensor emitter control (BE)	Front expansion
GP24	SPI1 RX, UART1 TX, I2C0 SDA, PWM4 A	—	Mid expansion
GP25	SPI1 CSn, UART1 RX, I2C0 SCL, PWM4 B	Yellow LED / Button A	—
GP26	SPI1 SCK, UART1 CTS, I2C1 SDA, PWM5 A, ADC0	Battery level input (VBAT/11), Line sensor emitter control (DNE)	Front expansion
GP27	SPI1 TX, UART1 RTS, I2C1 SCL, PWM5 B, ADC1	—	Mid expansion
GP28	SPI1 RX, UART0 TX, I2C0 SDA, PWM6 A, ADC2	—	Mid expansion
GP29	SPI1 CSn, UART0 RX, I2C0 SCL, PWM6 B, ADC3	—	Mid expansion
QSPI_SS_N	Flash chip select, BOOTSEL	Button B	—

Pin	Alternative RP2040-Funktionen	Standardfunktion am 3pi+	Header
RUN	Active-low reset	Reset button	Debug header
SWCLK	Serial Wire Debug (SWD): clock	—	Debug header
SWDIO	Serial Wire Debug (SWD): data I/O	—	Debug header

Pololu, [2025b](#)

B Bohrplan Labyrinth

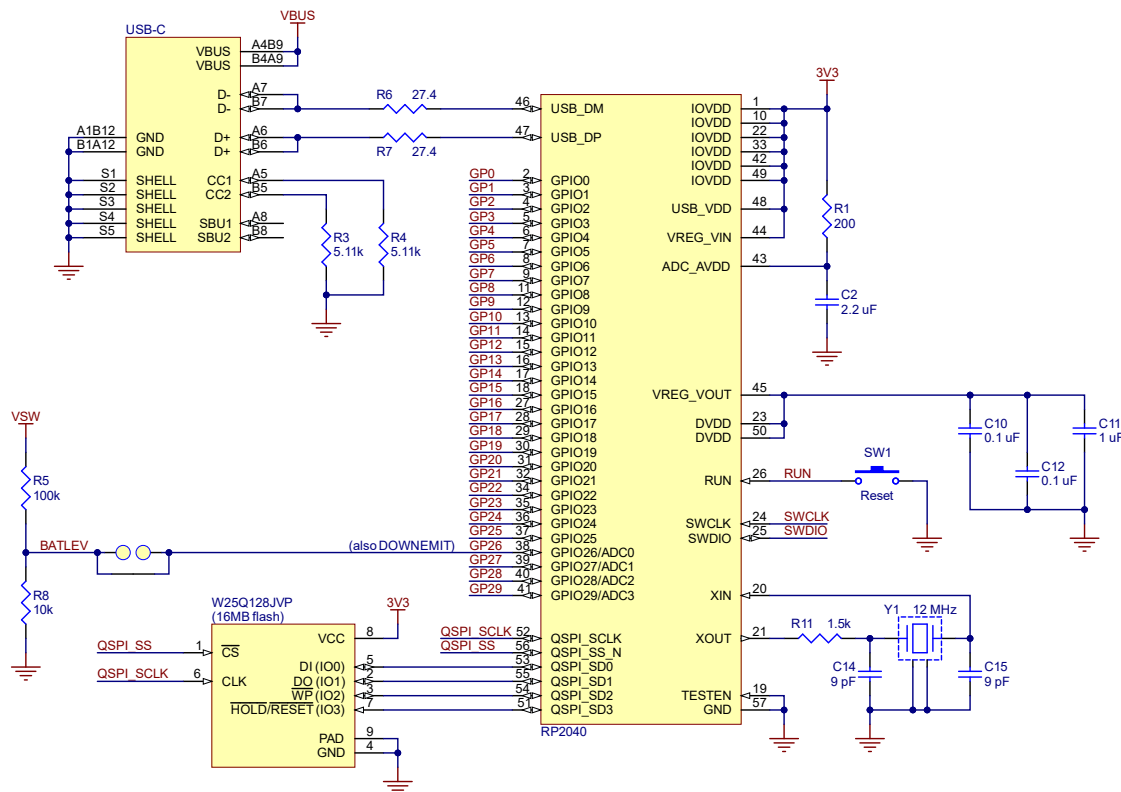


- Alle Maßangaben sind in mm
- Die Bohrungen sollen einen Durchmesser von 7 mm haben.
- Der Mittelpunktabstand zwischen den Bohrungen soll 180 mm betragen
- Der Abstand vom Rand zum Mittelpunkt der äußeren Bohrungen soll 50 mm sein
- Die Platte soll eine Stärke von 12 mm besitzen
- aus MDF

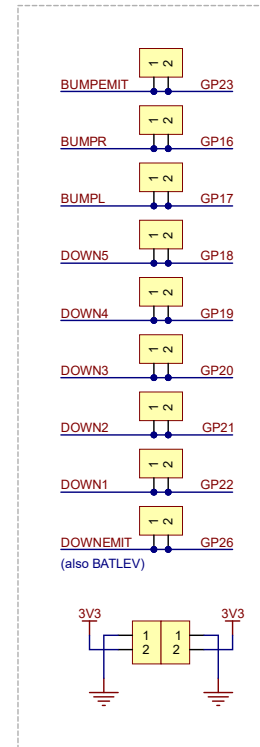
Abbildung 16: Bohrplan des Labyrinths

C Schaltplan 3pi+ 2040 Control Board

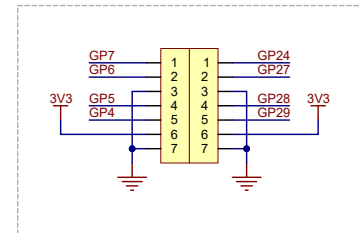
Auf den folgenden Seiten ist der Schaltplan des Control Board zu sehen.



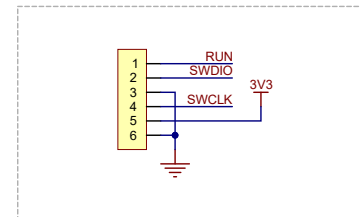
Front expansion header



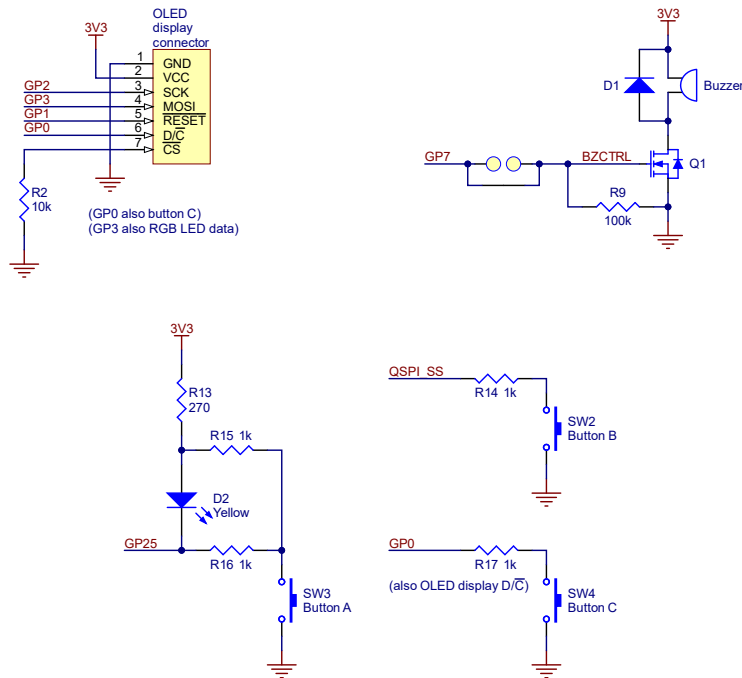
Mid expansion header



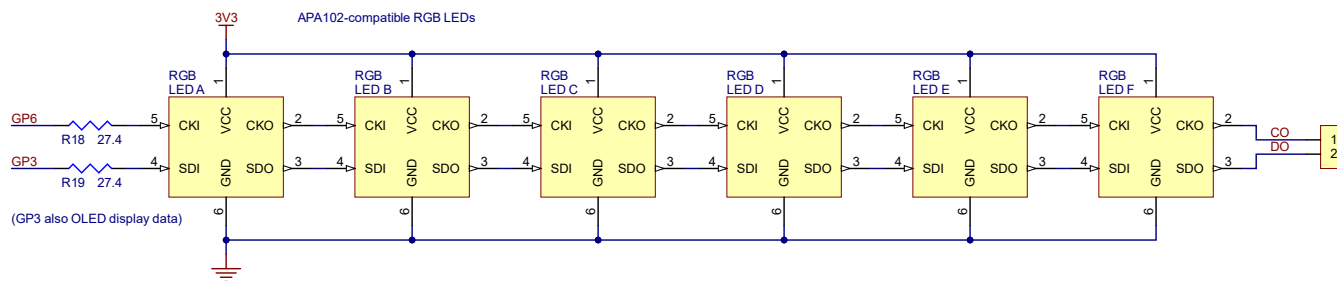
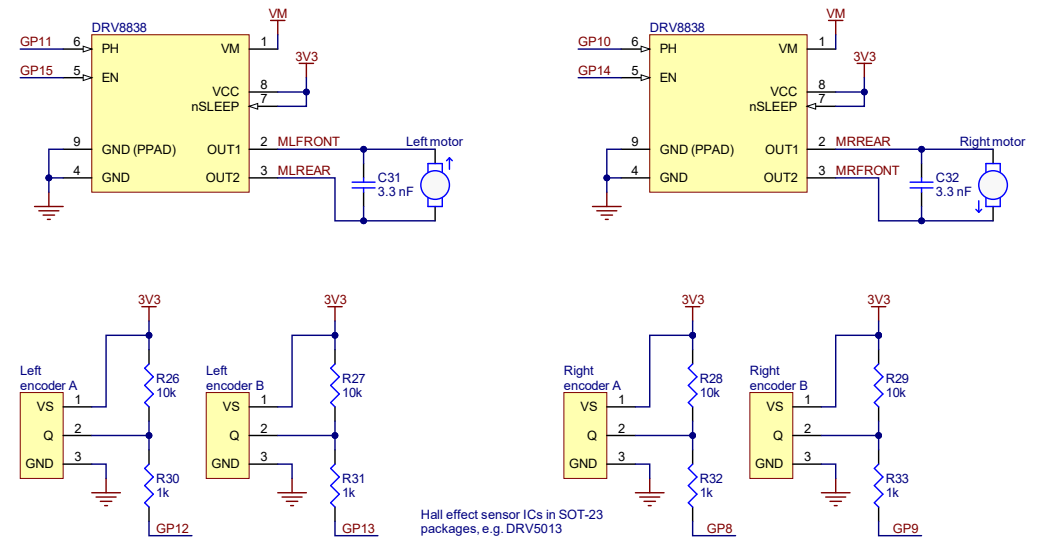
Debug header



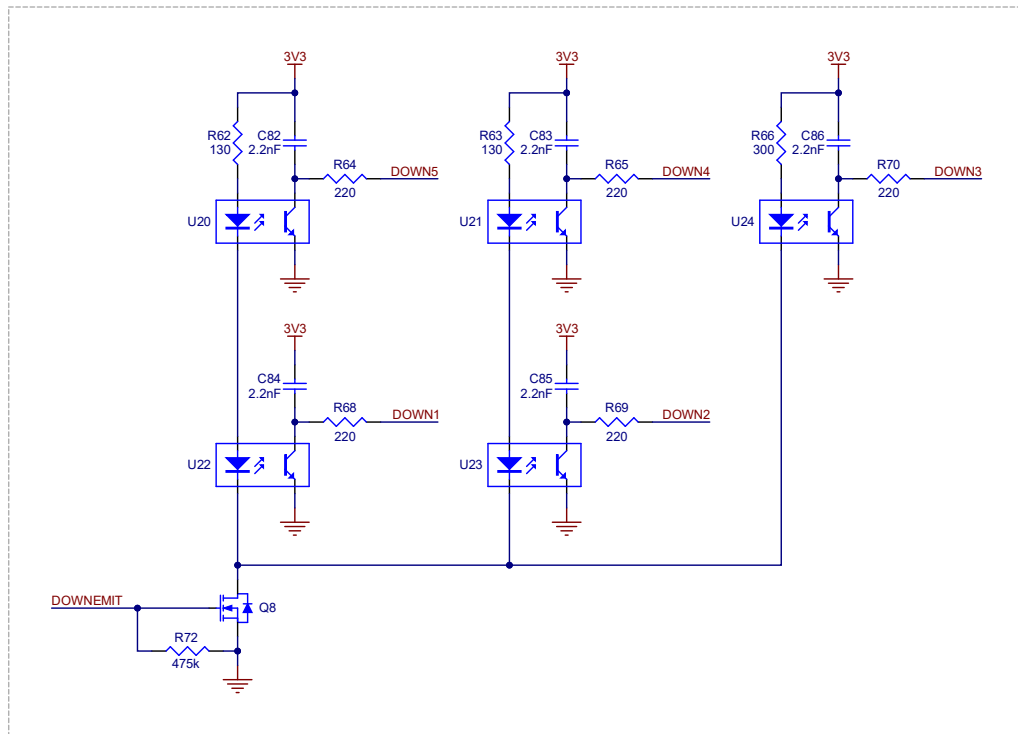
User interface



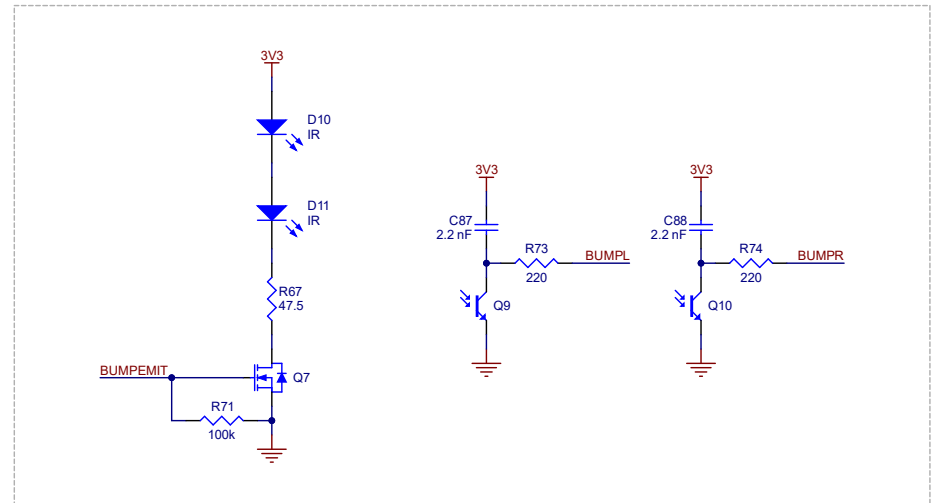
Motors and encoders



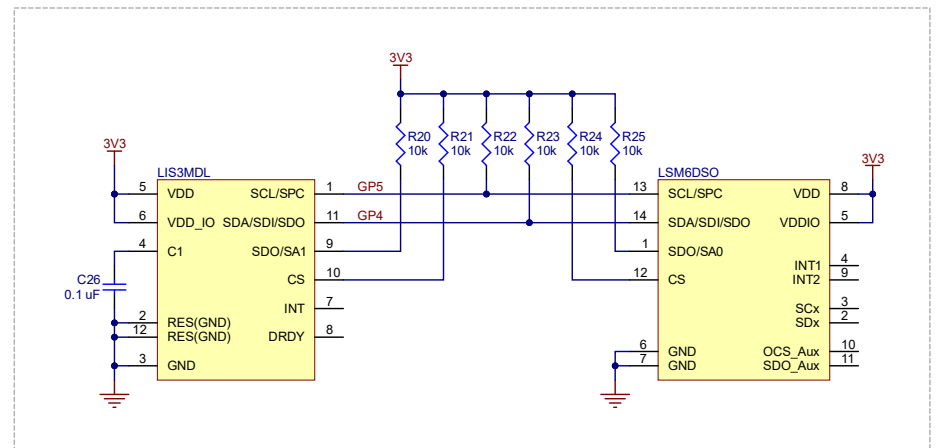
Line (reflectance) sensors

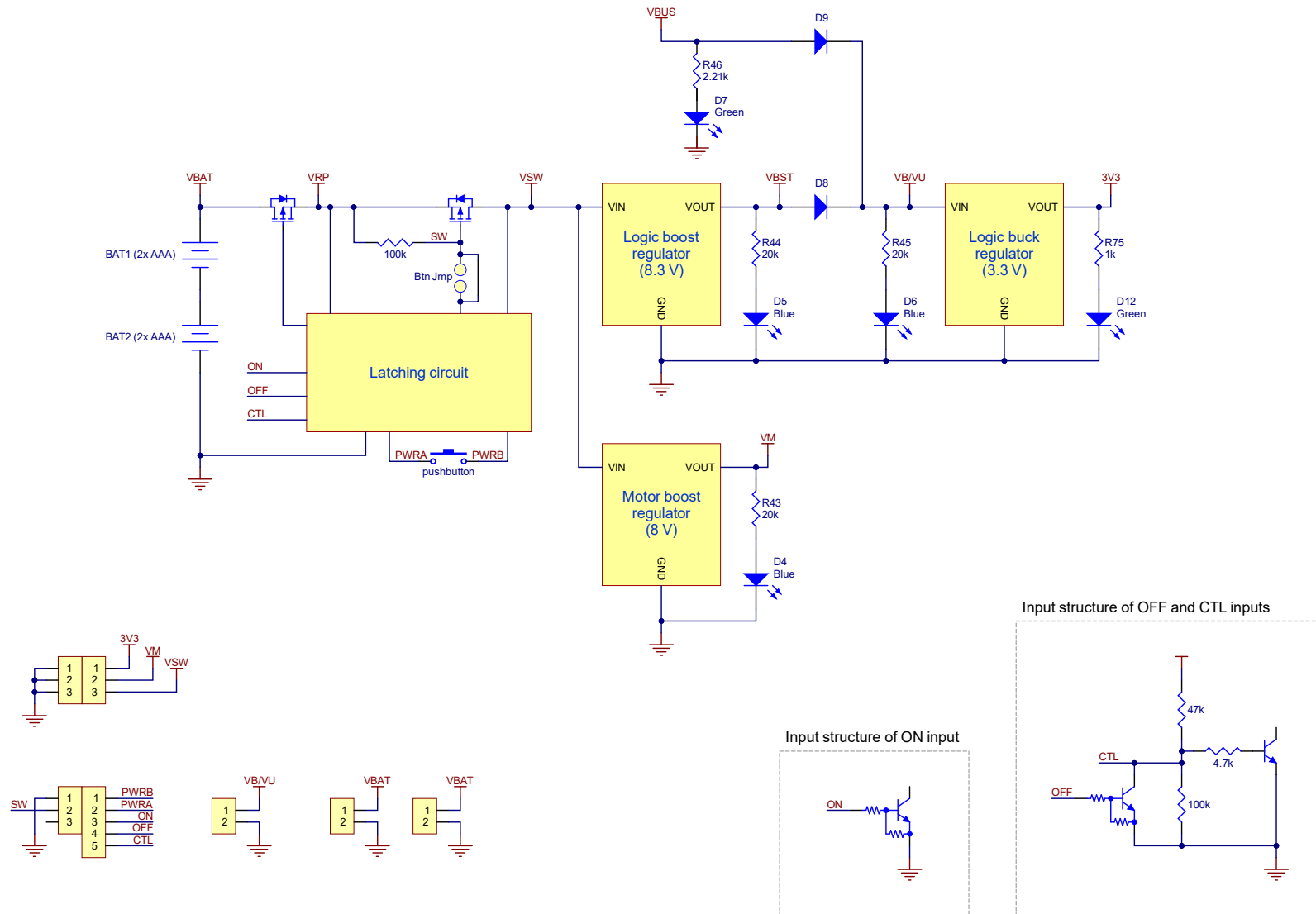


Bump sensors



Inertial sensors





D Laufzeitsensor (Time of Flight) OPT3101

Das vollständige Datenblatt für den in dieser Arbeit verwendeten Time-of-Flight-Sensor befindet sich als separate Datei `OPT3101_Datasheet.pdf` im digitalen Anhang.

- **Dokument:** High-Speed Time-of-Flight (ToF)-Based Long-Range Proximity and Distance Sensor Analog Front End (AFE)
- **Hersteller:** Texas Instruments
- **Dokumentnummer:** SBAS883A – FEBRUARY 2018 – REVISED JUNE 2018

E Schaltplan Pololu OPT3101

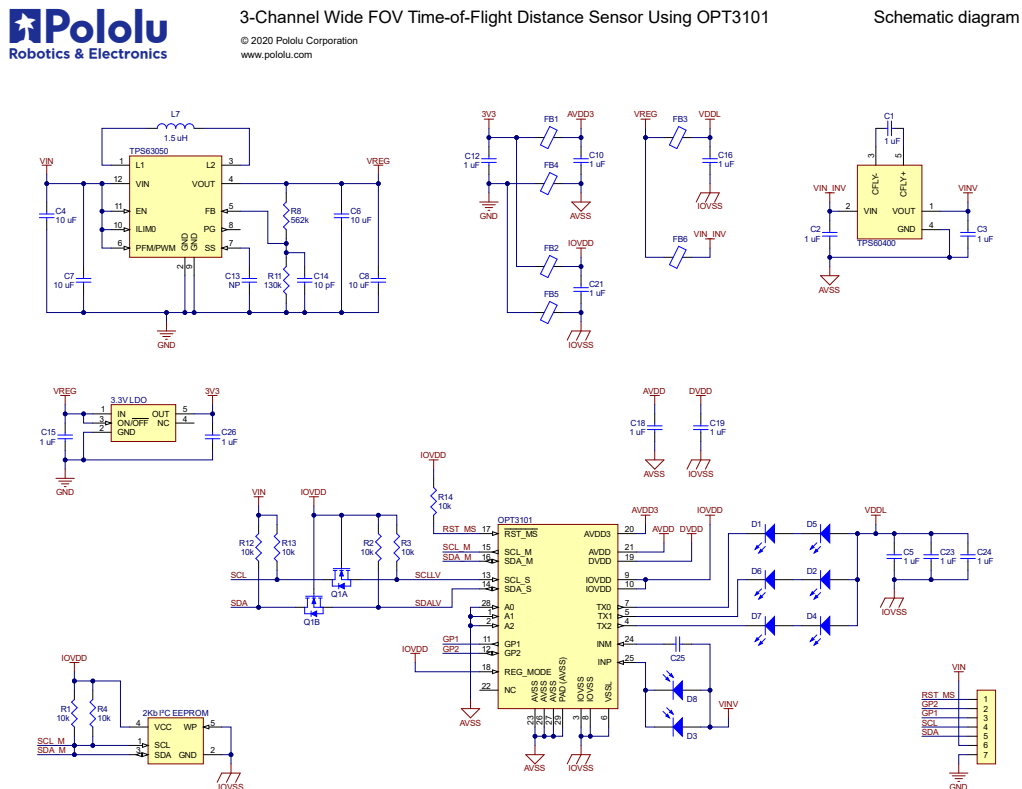


Abbildung 17: Schaltplan des verwendeten Distanzsensors.

F SH1106G OLED-Display

Das vollständige Datenblatt für den in dieser Arbeit verwendeten SH1106G OLED Displays befindet sich als separate Datei `SH1106_Datasheet.pdf` im digitalen Anhang.

- **Dokument:** SH1106 132 X 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller
- **Hersteller:** Sino wealth
- **Dokumentnummer:** SH1106G – FEBRUARY 2012 – REVISED JUNE 2013

G RP2040 Datenblatt

Das vollständige Datenblatt für den in dieser Arbeit verwendeten RP2040 Mikrocontroller befindet sich als separate Datei `RP2040_Datasheet.pdf` im digitalen Anhang.

- **Dokument:** RP2040 Datasheet - A microcontroller by Raspberry Pi
- **Hersteller:** Raspberry Pi

H GitHub-Bibliotheken

- [GitHub Bibliothek pololu-3pi-plus-2040](#)
- [GitHub Bibliothek Opt3101](#)
- [GitHub Bibliothek OLED Display](#)

I 3D-Modelle für die Labyrinthwände und -ecken

Die vollständigen STL-Dateien befinden sich im digitalen Anhang unter dem Ordner `stl files`. Alternativ können sie auch [hier von der Plattform Thingiverse](#) heruntergeladen werden.

J Weiterführende Links zur RP2040-Programmierung in Python

- [Raspberry Pi Pico Python SDK \(offiziell\)](#)
- [Dokumentation für MicroPython](#)
- [Kurzreferenz für den RP2040 unter MicroPython](#)
- [Dokumentation der RP2-Bibliothek in MicroPython](#)
- [Quellcode von MicroPython auf GitHub](#)
- [Die offizielle Python-Sprachreferenz](#)
- [Die Python-Standardbibliothek](#)

K Weiterführende Links zur RP2040-Programmierung in C++

- [Anleitung: Erste Schritte mit dem Raspberry Pi Pico](#)
- [Dokumentation des Raspberry Pi Pico SDK](#)
- [Quellcode des Raspberry Pi Pico SDK auf GitHub](#)
- [Offizielle Code-Beispiele für das Pico SDK](#)